Client internals and version profiles, part 4 of 4

Client internals and version profiles, part 4 of 4

This part contains 73 notes, from `client-internals/offline-training.md` through `client-internals/decomp-findings-2026-10-28-b/01-tile-type-tests.md`.

`client-internals/offline-training.md`

Offline training levels

Date: 2026-10-14

Derived from observed client behavior. No decompile material.

Observation

Tutorial levels are embedded in the executable. They can be drawn and walked without a network connection. The client does not require a listserver or gserver to enter them.

Difference from session levels

The global that holds the current level name is set only by the session “set level” message (the byte that arrives after a successful world login). Offline training levels never set that global. A debugger session that shows a training level drawn while the level pointer remains null is expected and correct.

Enter path

Entering a training level stores the level name on the player object and the arrival coordinates. Nearby links (for example to a house interior) are present in the embedded data. The same collision and script paths used by disk levels are used here.

Offline implication

An offline session that loads a GLEVNW01 file from disk is already past the embedded-tutorial case. The same collision, sign, and script paths apply. The only difference is the source of the level data (disk versus embedded).

The offline slice therefore does not need to special-case training levels. It only needs to:

1. Load a level from disk (already done).
2. Apply the tile walk rules (see `formats/tile-walk.md`).
3. Execute the common GS1 commands on touch (see `npc-library/gs1-commands.md`).
4. Blit the selected sheet (see `rendering-observations.md`).

Once those four are in place, both embedded training levels and disk levels will behave the same way.

See `offline-tutorial-205.md` for the 2.05 details: the *Play offline* path, the resource lookup, and the level-by-level contents.

`client-internals/offline-tutorial-205.md`

Offline mode and the embedded tutorial (client 2.05 rev 2)

Date: 2026-10-21

Derived from the 2.05 rev 2 binary (resources, strings, and the offline and level-load paths) and from decoding the embedded levels with `formats/grv1-decoder/grv1.py`. Names only, no code. Tags: **[2.05]** · **[2.22]** · **[inf]**.

This note refines `offline-play.md` and `offline-training.md`.

How the client goes offline

Offline play is reached in these ways:

1. **Training on the start screen.** It starts the embedded tutorial at `tutorial_b-4.graal` with no connection (confirmed live, see `live-checks-editor-offline-sound.md`). **[2.05]**
2. **"&Play offline" on the reconnect dialog** (`TReconnectFrame`). The dialog appears when the connection fails or drops ("The connection could not be established!", "You have been disconnected from the game server.", "There are too many players online."). Choosing *Play offline* frees the game socket, so the connection object is nil from then on. **[2.05]**
3. **Command line.** `Graal.exe level.graal` starts with a local player and opens that file in the editor. Play then runs it offline (see `level-editor-205.md`).
4. **F5 Load level** while online. The client warns "When you load a level you will be disconnected from the server. Continue?" and then loads the file locally.

"Offline" in the client simply means *no game connection object*. Every check that refuses the editor, enables the tutorial resources, or sends file requests tests that one condition.

Where the player lands

When *Play offline* is chosen:

- If the level the player was on exists in the local folder, the player stays on it at the same coordinates.
- Otherwise the player is moved to **`tutorial_b-4.graal` at (32, 60)**. The map overlays are switched to the tutorial set: `tutorialmap.png` / `tutorialmap.txt` for the big map and `tutorialmap2.png` / `tutorialmap.txt` for the minimap. A "first offline start" flag is also set.
- If the level still fails to load, a warning names the level and the client stays at the reconnect dialog.

Both of the first two cases were seen live: after a refused connection the player went to the tutorial, and after a drop on a level that was also on disk the client reloaded that level at the same spot (see `live-checks-editor-offline-sound.md`).

Where offline levels come from

The level loader checks two sources in order. **[2.05]**

1. **Embedded tutorial.** When offline, and the name starts with `tutorial`, the loader maps the file name to an `RCDATA` resource and reads the level from the executable.
2. **Disk.** Otherwise it looks the name up through the file-name cache and reads the file from disk.

The embedded copy wins: with a `tutorial_b-4.graal` of different content in the game folder, the client still loads the embedded level and does not look for the file (live).

Online, names starting with `tutorial` are not special. They go to the server like any other level.

Level file	Resource
`tutorial_a-1.graal` … `tutorial_a-4.graal`	`TUTORIALA1` … `TUTORIALA4`
`tutorial_b-1.graal` … `tutorial_b-4.graal`	`TUTORIALB1` … `TUTORIALB4`
`tutorial_b-2_house.graal`	`TUTORIALB2H`
`tutorial_c-1.graal` … `tutorial_c-4.graal`	`TUTORIALC1` … `TUTORIALC4`

All 13 are `GR-V1.03` binary levels (see `formats/level-binary.md`).

Special level names

Three level names trigger an extra setup step when loaded: `level1.graal`, `YoshiClanR2.zelda`, and `YoshiClanFields.zelda`. These are the original single-player start levels. What the step does has not been traced. **[2.05]**

The tutorial world

The levels form a 3×4 grid: letters are columns from west to east, digits are rows from north to south. Edge links use `playerx` / `playery` to keep the other coordinate, landing at 0 or 61 on the far side.

```
      a        b                    c
1   a-1  ──  b-1                ──  c-1
     │        │                    │
2   a-2  ──  b-2 (+ b-2_house)  ──  c-2
     │        │                    │
3   a-3  ──  b-3                ──  c-3
     │        │                    │
4   a-4  ──  b-4  START (32,60)  ──  c-4
```

Level	Links	NPCs	Chests	Signs	Content
a-1 … a-4, c-1 … c-4, b-1	2–3	0	0	0	Scenery only
b-4 (start)	3	8	1	3	Welcome sign ("In this tutorial you will learn … say the password 'Balibu'"). A gate NPC opens when the player chats `Balibu` and stands in a y range. A `setmap tutorialmap.png` … / `setminimap` … NPC. Two light NPCs (`drawaslight`, `setcoloreffect`). Two character NPCs. The chest holds `glove1` (the red power glove) with sign 0: "Go to the stone, press … to lift it!"
b-3	4	0	0	1	Sign at (36, 37): "Use the sword to cut those trees!"

Level	Links	NPCs	Chests	Signs	Content
b-2	5	5	0	2	An arrow box NPC (`lay darts`). A door NPC that opens on the `opendoors` flag and plays `chest.wav`. Character NPCs. The link into the house is at (22, 26). The signs explain shooting statues to open the door, then "Excellent! You got the treasures! … Press F8!"
b-2_house	1	6	6	7	A shopkeeper ("give me 5 rupees", how to drop a rupee). A house gate NPC that slides open on the `housegateopen` flag with `stonemove.wav`. Two lamps, two lights. Six chests: battleaxe, lizardshield, spinattack, nukeshot, fullheart, superbomb, each with its own sign.

The tutorial ends by sending the player to **F8** (server list) to go online.

Signs at (0, 0) are not drawn on the board. Chests and the `say n` command show them by index. Six of the seven house signs are chest signs.

Later clients

- **2.17, 2.22 and 2.31** keep embedded tutorials but trim them to a 3×3 grid of 9 levels named `offlinetutorial_a-1.graal` … `offlinetutorial_c-3.graal` (resources `TUTORIALA1` … `TUTORIALC3`, still `GR-V1.03`). Training starts on `offlinetutorial_b-2.graal`, the centre of the grid (2.22 and 2.31, live). **[2.17][2.22][2.31]**
- Offline, every client tested (2.05, 2.22, 2.31) reads `levels/tiledefs/tiledefsOffline.txt`, because the tile-definition file name is `tiledefs` + server name + `.txt` and the offline server name is `Offline`. The shipped file only targets `offlinetutorial*` levels, so it has no effect on the 2.05 tutorial. **[live]**
- The standalone GraalEditor 2.3.1.1 carries the same 9 `offlinetutorial_*` names and `tiledefsOffline.txt`. **[inf from strings]**

Offline implications

- An offline reimplementation that wants the original first-run experience needs the 13 resources and the `tutorial*` resource lookup. Decoded copies can be produced locally with the decoder. The levels are not committed here.
- The resource mapping applies only when the client has no connection, so a server can host its own `tutorial_*.graal` levels without conflict.

`client-internals/level-editor-205.md`

Built-in level editor (client 2.05 rev 2)

Date: 2026-10-21

Derived from the 2.05 rev 2 binary: class names (RTTI), form resources (DFM), string tables, and the behaviour of the editor handlers. Names only, no code. Tags: **[2.05]** read from the binary · **[inf]** inferred.

This note covers 2.05. The primary editor reference is now `tools/level-editor/222-reference.md` (2.22), with the other builds as deltas in `tools/level-editor/version-deltas.md`.

In 2.05 the editor is not a separate program. It lives in the main window (`TAdventure`) and is a second mode of the game screen. GraalEditor.exe split off later; see `tools/graaleditor-vs-205.md`.

Entering and leaving

Key	Game mode	Notes
F4	open the editor	Refused when connected: "You cannot open the editor in the online mode." **[2.05]**
F5	Load level dialog (`TLoadLevel`)	Asks first when online: loading a level disconnects you.
P / Play button	back to game mode on the current level	A named, modified level is saved first ("Level saved"). "Please save the level under a new name before playing!" if it has no name.
Close button	close the current level tab	Asks "Save changes to …" when modified.

Entering the editor tears down the game state for the current level: looped sounds stop, NPC and baddy instances are freed, and the level the player stood on is opened as a tab (re-read from disk, so unsaved runtime board changes are not carried over). The scroll position centres on the player. **[2.05]**

Starting the client with a level file as the first command-line argument (`Graal.exe path\level.graal`) skips the login screen. The client changes into the file's folder, creates a local player at (30, 30.5), and opens the file in the editor. **[2.05]**

Window layout

- **Board view**: the DirectDraw surface shows the 64×64 level at 16 px per tile, with scroll bars.
- **Toolbar**: New, Open, Save, Save as, *Undo board copy*, Cut, Copy, Paste, Delete, *Turn selection into a warp field*, Edit links, Edit signs, Edit flags, Close, Play.
- **Tab bar**: one tab per open level. Each tab keeps its own scroll position, selection, floating paste, undo stack, and object lists. New levels are titled `(new)`.
- **Side panel**, with three radio modes:
    1. *Select predefined object*: stamps from GOBJSET01 sets (see below).
    2. *Select tile*: the tileset palette (`pics1.png`). Click to pick, drag to pick a block. Double-click a tile to make it the background tile shown in the preview box.
    3. *Baddies*, *Npcs*, *Chests*: a palette of placeable objects.
- Checkboxes: *Show blocking tiles*, *Show foreground tiles*, *Show terrain tiles* (overlays computed from the tile-type table, not stored in the level).
- Status bar: tile coordinates, tile index, and the object under the cursor (`tile`, `npc`, `chest`).

Editor keys

The editor uses Delphi shift states. Ctrl is the main modifier.

Key	Action
Ctrl+N	New level
Ctrl+O	Open
Ctrl+S	Save

Key	Action
Ctrl+Z, Alt+Backspace	Undo board copy
Ctrl+X, Shift+Del	Cut
Ctrl+C, Ctrl+Ins	Copy
Ctrl+V, Shift+Ins	Paste
Del	Delete selection
L	Edit links
S	Edit signs
P	Play
M	Generate a map image (`graalmap.png`, "Generating the map may take some minutes. Proceed?"). It walks the level list of the loaded big map (`bigmap.txt` or the list a `setmap` NPC loaded), 56×56 px per level, and does not follow links.

L, S, P and M are ignored while a combo box in the side panel has focus.

Tile editing model

- **Selection.** Dragging on the board selects a tile rectangle. The selection is clipped to the 64×64 board.
- **Floating paste.** Copy or Paste lifts the selected tiles into a floating block that follows the mouse. Clicking outside it drops it. Dropping writes the tiles onto the board. Moving a selection works the same way: it is lifted, dragged, and dropped.
- **Right mouse button** on a placed baddy, NPC, or chest makes a copy of it ("Right Mousebutton-> copy …").
- **Warp field.** *Turn selection into a warp field* creates a link whose rectangle is the selection and opens the link dialog.

Undo

Undo covers **board tiles only**, which is why the button says *Undo board copy*.

- Before any write to the board, the editor stores an undo record: the rectangle (x, y, width, height) plus the tile values it is about to overwrite.
- Records are kept per level tab in a stack of at most **128**. The oldest record is dropped when the stack is full.
- Undo pops the newest record, writes the old tiles back, re-selects that rectangle (or clears the selection if it was the whole board), and marks the level modified.
- Adding, moving, or deleting links, signs, NPCs, baddies, and chests is **not** undoable.

Object types

The editor's level document (`TAdventureLevel`) keeps separate lists, one per kind.

Kind	Class	Placed with	Dialog (double-click)
Link	`TEditorLink`	warp-field button, or the link list	`TLinkList` → `TLinkOptions`
Sign	`TEditorSign`	sign list "New"	`TEditSigns`
Chest	`TEditorChest`	object palette	`TChestEdit`
NPC	`TEditorNPC`	object palette	`TEditPerson` → `TPersonAction`
Character NPC	`TEditorNPC` with a generated script	object palette	`TEditCharacter`
Baddy	`TComputer`	object palette	`TEditBaddy`
Stamp	`TEditorObject`	"Add the selection to the objects"	`TObjectName`, `TObjectSet`

Links (`TLinkList`, `TLinkOptions`)

A list with columns Linkfile, X, Y, W, H, New X, New Y, and buttons Edit, Delete, Load. *Load* opens the destination level in a new tab. The link dialog accepts a file name or URL, and the new position may be an expression: "you can use math. operations and variables like playerx/y". The tutorial levels use `playerx` and `playery` this way.

Signs (`TEditSigns`, `TShowCodes`)

A list of signs with X and Y fields, New, and Delete. *Show special codes* lists the glyph escapes (`#h` head, `#A`, `#B`, …). A sign at (0, 0) is not drawn on the board. Such a sign is used by index: a chest shows sign *n* when opened, and the GS1 `say n` command shows sign *n*. The embedded tutorial uses both.

Chests (`TChestEdit`)

The chest holds an item and a sign index (-1 = no sign). The 2.05 item list, in index order:

```
greenrupee, bluerupee, redrupee, bombs, darts, heart, glove1, bow, bomb, shield, sword,
fullheart, superbomb, battleaxe, goldensword, mirrorshield, glove2, lizardshield,
lizardsword, goldrupee, fireball, fireblast, nukeshot, joltbomb, spinattack
```

A new chest defaults to item 1 (bluerupee) and sign -1.

Baddies (`TEditBaddy`)

There are 10 types: Gray Soldier, Blue Soldier, Red Soldier, Shooting Soldier, Swamp Soldier, Frog, Spider, Golden Warrior, Lizardon, Dragon. The script names are `graysoldier` … `dragon`, and the sprites are `baddy*.gif`. Each baddy has three verses: *attack*, *when hurt*, and *when it hurts the player*. A new baddy starts with "Stop!" and "Ouch!". Types 4–6 (Swamp Soldier, Frog, Spider) do not talk, and the writer saves empty verses for them.

NPCs (`TEditPerson`, `TPersonAction`)

An NPC has a GIF image (or none) and a GS1 script. The script editor has *Test* (syntax check, with the error shown in a status line) and *Style*, plus the usual Undo/Cut/Copy/Paste menu. Runtime flags set by scripts while testing appear in *Edit flags* (`TEditFlags`). That dialog edits the in-memory flag list, not anything stored in the level. The list survives Play and a return to the editor, but it is never written to disk and is lost on exit (live).

Character NPCs (`TEditCharacter`)

A form generates a script that sets up a walking character. The fields are character, shield, nickname, horse, direction, walk mode (None/Random/Left-Right/Up-Down/Rectangle), speed, attack mode (None/Sword), hunt speed, sword power, attack verse, power, roaming box X1–X2/Y1–Y2, respawn time, and the flags *can be carried*, *sword hurts*, *bomb/arrow/stone hurts*, and *attack other npcs too*. The output starts with `// Graal v1.4 NPC by Stefan Knorr` and `showcharacter;` / `setcharprop` lines. After that the NPC is an ordinary NPC: double-clicking it opens the normal NPC editor. **[2.05]** Several tutorial NPCs were made this way.

Stamps (predefined objects)

Object sets are read from every `objects*.txt` file listed in the file-name cache. The set name is the part between `objects` and `.txt`, and plain `objects.txt` is the *Standard* set. Each file is `GOBJSET01` (see `formats/gobjset01.md`). *Add the selection to the objects* names the current selection (`TObjectName`) and inserts it after the selected stamp. *New* (`TObjectSet`) creates `objects<setname>.txt`. *Save objects* rewrites the set file.

Saving

The file extension picks the format. **[2.05]**

- **`.nw`**: GLEVNW01 text. It writes `GLEVNW01`, then 64 `BOARD 0 y 64 0 <base64 tiles>` rows, then `LINK …`, `SIGN x y … SIGNEND`, `CHEST …`, `BADDY x y type` + three verse lines + `BADDYEND`, and `NPC image x y … NPCEND` (image `-` when there is none).
- **Anything else** (normally `.graal`): always `GR-V1.03`. The 2.05 editor cannot write older GR versions.

GR-V1.03 writer details, which complement `formats/level-binary.md`:

- The board is written with 13-bit codes. The encoder emits a single run (`count` ≤ 64) for two or more equal tiles, and a **double run** (`ABAB…`, ≤ 64 pairs) when the pattern `A B A B` starts at the cursor. Otherwise it emits a plain tile. So 2.05 does write double runs, which answers one of the open questions in that file.
- Baddy records are `x, y, type` as raw bytes (not +32), then `verse1\verse2\verse3\n`. The section ends with `FF FF FF 0A`.
- NPC records are `x+32, y+32, image#script\n`. Chest records are four GCHARs. Sign records are two GCHARs plus sign-alphabet text.
- Saving a level that was opened from a URL (`http://…`) falls through to *Save as*.

Opening accepts `Z3-V1.03`, `Z3-V1.04`, `GR-V1.00`–`GR-V1.03`, and `GLEVNW01`. Anything else gives "Unsupported file type!".

Loader note for formats/level-binary.md

The 2.05 loader picks the code width by magic: **13 bits only for `GR-V1.02` and `GR-V1.03`, 12 bits for `GR-V1.00`, `GR-V1.01`, and `Z3-V1.*`.** The section order also depends on the magic: NPCs from `GR-V1.00` on, and chests from `GR-V1.01` on. The baddy section ends at the first record shorter than three bytes or whose first byte is ≥ 0x80. It does not need a literal `FF FF FF`. An x byte of 0 is a valid baddy, not a terminator (corrected 2026-10-25; the loader has no x = 0 test in 1.39, 2.00, 2.05, 2.17.1 or 2.22; see `decomp-findings-2026-10-25-f/03-baddy-terminator.md`). **[2.05]**

Open

- *Generate map* and *Edit flags* were answered live; see `live-checks-editor-offline-sound.md`.

Batch notes

Dated decompile-finding batches. Each batch stays together, and the batches are in date order. The topic notes above are the place to start; these are the passes those notes came from.

`client-internals/decomp-findings-2026-10-24/README.md`

Decompile findings, 2026-10-24 (derived facts only)

Built from three Ghidra decompiles (2.05 rev2, Graal 1.39, 1st Graal). Nothing here quotes code or addresses.

File	Topic
01-death-respawn.md	zero-hearts handler, respawn, damage rules
02-sprites-poses-emotes.md	sprite prop, poses, swim, emotes
03-player-list-popup.md	F7 list, popup, PM recipients, TPersonAction
04-sword-hitbox.md	sword swing and hit evaluation
05-205-props-misc.md	other 2.05 facts
06-first-graal.md	1st Graal protocol summary
07-graal-139.md	1.39 protocol summary
08-contradictions.md	new facts and contradictions

Tables (TSV): sprite-step-table, gani-to-step, gani-name-table, gani-short-codes, status-bits-prop18, player-modes, life-states, constants-205.

Gaps still open

- Wire representation of swim (none found) and how others see emotes.
- Damage formula (inferred sword power / 2).
- Meaning of a few unlabelled mode values. (Status bit 0x20 is now answered: `../status-bit-0x20.md`.)
- Exact "same name group" rule for the saved respawn point.
- 1st Graal room-name branch selection; 1.39 popup, death and sword.

`client-internals/decomp-findings-2026-10-24/01-death-respawn.md`

2.05 rev2: death and respawn (derived)

Source: static analysis of the 2.05 rev2 client, 2026-10-24. Behaviour only; no code is reproduced. Tick is 50 ms. See `player-modes.tsv`, `life-states.tsv` and `constants-205.tsv`.

Trigger

- Any change that leaves hearts at 0 or below starts the death sequence once.
- A per-frame safety net also forces the dead life-state whenever hearts are 0 or below.

Sequence on the dying client

1. A death timer starts at 0 and counts up each tick. It stops at 30 (1.5 s).
2. Life-state becomes "dead", mode goes back to 0 (normal) and the player faces down.
3. The "dead" gani plays for the local player and the script event `playerdies` fires.
4. Controls are ignored until the timer passes 27 (1.35 s).
5. Status prop (18) gets the dead bit (0x08) only when life-state is dead and the timer is at least 28. So other players see the player as dead only after about 1.4 s.
6. No sprite for "dead" goes on the wire. The sprite prop shows plain idle plus direction. The dead state is carried by the status bit only.

Respawn

- After the timer passes 27, any movement or action key triggers the respawn. There is no dedicated respawn packet and no server permission step on this path.
- **Hearts after respawn** (AP = alignment points):

AP	Hearts
below 20	3
20 to 39	min(5, max hearts)
40 or more	max hearts

- **Destination** is the last saved level-entry point (level, x, y, direction). It is updated on every level entry, unless the old and new levels share the same name group (see the gap list), and also at login.
  - If nothing was saved and the client is online: x 30.5, y 30.0, facing down, in the current level.
  - Offline: the default start level `tutorial_b-4.graal`.
  - When the online flag is set, the client re-sends the current level at the current x/y, so the server sees an ordinary level warp (the PLI_LEVELWARP route) rather than a "died" message.
- After respawn the status bit 0x08 clears when life-state returns to playing.

Damage rules that feed this

- Healing only happens at AP 40 or above.
- No damage in swim mode (10) or in the editor/pause state unless forced.
- Hurt state: mode 3 for 40 ticks, "hurt" gani, knockback, script event `playerhurt`. A player already in mode 3, mode 10 or dead cannot be hurt.
- The victim's client decides hits (see `04-sword-hitbox.md`). When hearts reach 0 or below after a sword hit, an explosion hit or a UDP hurt, the victim client sends PLI_CLAIMPKER (14) carrying the attacker's player id.

Anti-cheat note

Hearts, AP and max-power sit in an attribute array that the client stores obfuscated twice and cross-checks. A mismatch raises "memory problems error 1/2". Editing hearts from outside the client is therefore detected by the client itself.

Gaps

- The exact "same name group" rule for not updating the saved entry point was only partly traced.
- The server-side reaction to CLAIMPKER is not in the client binary.

`client-internals/decomp-findings-2026-10-24/02-sprites-poses-emotes.md`

2.05 rev2: sprite index, poses and emotes (derived)

Tables: `gani-name-table.tsv`, `gani-short-codes.tsv`, `gani-to-step.tsv`, `sprite-step-table.tsv`, `status-bits-prop18.tsv`, `player-modes.tsv`.

How the sprite prop (17) is built

- Wire value = 32 + direction + 4 x S.

- S comes from the current animation step through the step table, only when the active gani is the default "def" gani and the step is 38 or lower.
- Hurt is special: S = 32 (value 0x80 + direction as seen in the byte).
- Other ganis do not change the sprite number. (Inferred: they travel by the gani prop.)
- Each gani name maps to a step by a small rule list (`gani-to-step.tsv`): walk is frame+1, sword/ridesword/spin are 9 + frame mod 5, push 14+frame, pull 20+frame, grab 19, lift 23, carry still 24, carry 25+frame, shoot 33, ride variants 34 to 36, sit 37, sleep 38, hurt 39, dead 40.
- Caveat: the "only for def" condition was read from the sender and is documented as observed. The same sender sets everything to 0 for unknown names.

Swim, lava and dead on the wire

- There is **no swim sprite on the wire**. Swim, lava and dead are all sent as sprite 0 (idle) plus direction.
- Swim is local mode 10, entered on tile type 11 with a splash. Local gani "swim" is name-table index 28 and "lava" is index 29, but neither has a sprite value.
- Tile type 12 shows the "lava" gani.
- Inferred: other players can tell a swimmer apart only through the gani prop, not the sprite.

Gani name table

31 names, index 0 to 30 (`gani-name-table.tsv`). The short-code table (`gani-short-codes.tsv`) maps substrings to one letter and is used to shorten cached gani file names.

Emotes (Ctrl+letter)

- Holding Ctrl and pressing A to Z (while no text field has focus and the player is not dead/locked) calls the emote routine with that letter.
- The routine lowercases the letter and walks the loaded file list. It takes the **first file whose name starts with** `emoticon_<letter>` and whose extension is png, mng or gif. (A prefix match, so Ctrl+A matches `emoticon_afk_stay` if that file is first in the list.)
- The image is placed at the player's x+2, y-5 (a little lower if a second flag is set) and shown for 50 ticks (2.5 s).
- The emote is data driven. No letter-to-name table exists in the client, and there are no hard-coded AFK/BRB/Zzz strings.
- **No network send was found on the key path.** The PLI_SHOWIMG (24) sender is reached only from the script command `showimg`. How other players see an emote (the captured AFK image) is therefore not explained by the key handler. Treat that as open.
- The same routine and file-name scheme exist in 1.39, so emotes are not new in 2.05.

Status prop (18)

See `status-bits-prop18.tsv`. Dead (0x08) is delayed as described in `01-death-respawn.md`.

`client-internals/decomp-findings-2026-10-24/03-player-list-popup.md`

2.05 rev2: F7 player list, right-click popup, person action (derived)

Player list window

- Stay-on-top form with a multi-select list view. A panel caption reads "--- N online ---". Buttons: Profile, Options.
- Popup menu items in order: **Message** (default item), **History**, **Buddy**, **Ignore**, **Profile**.
- Right-click selects the row under the cursor, then sets the Ignore and Buddy check marks to "checked if all selected players are in that list".
- Double-click or Enter: opens the read-PM window if a PM from that player is pending, otherwise the compose window.

Actions

Item	Behaviour
Message	Builds the recipient list from all selected rows and reuses an existing PM window for the same set. Recipient encoding: a count as two bytes, then two bytes per player id, at most 50 (0x32) recipients.
History	Opens the message history window (not traced further).
Ignore	Toggles each selected account in the ignore list and saves `ignoredaccounts.txt` and the buddy list.
Buddy	Toggles the buddy list (`buddyaccounts.txt`). Ignore and Buddy are mutually exclusive.
Profile	Uses only the first selected player. If online, sends PLI_PROFILEGET (80).

- Messages from ignored players get the auto reply string "Your messages are ignored." (the string exists; the send condition was not traced).
- PM window titles: "Send Message" and "Read Message".
- Options window: 5 check boxes. Profile edit window: 10 fields.

"Person action" is not a player menu

`TPersonAction` is the "Set Npc's Action" script editor used by the level editor. It has nothing to do with right-clicking a player. There is no per-player action form (trade, duel and so on) in the 2.05 client.

Not found

- No right-click handler on a player sprite in the game world was found in 2.05.

`client-internals/decomp-findings-2026-10-24/04-sword-hitbox.md`

2.05 rev2: sword swing and hit box (derived)

Starting a swing

Preconditions: weapons allowed by the level or server, sword power not 0, not swimming without a horse, and not already in the charged-spin mode. - Normal swing: mode 6. Charged spin: mode 13 (0xd); needs the has-spin flag.

- Plays the sword, ridesword or spin gani and a sound, then checks for a cuttable bush at the sword tip. - The sprite sent to others follows steps 9 to 13 (sprites 6 to 10) of the swing; see `02-sprites-poses-emotes.md`.

Who decides a hit

The **victim's client** evaluates the hit and applies damage. The attacker only broadcasts its position, direction and sprite. A hit counts when all of these hold: - the attacker is on step 9 to 13 (sprites 6 to 10); - the victim is 0 to 4 tiles ahead of the attacker in the facing direction; - the sideways offset is under 2 tiles; - the victim's AP is not exactly 100 (such players cannot be hit); - the victim is currently hurtable (not in mode 3, mode 10, dead or editor).

Damage

Inferred: sword power divided by 2 in hearts. Not confirmed from a capture.

Explosions

A bomb hit needs the victim within 1 tile on both axes of the blast, with knockback of 0.5 or 1.0 tile.

Gaps

- Exact damage formula and the timing of the swing frames in milliseconds are not confirmed; frame durations come from the gani files, not the client.

`client-internals/decomp-findings-2026-10-24/05-205-props-misc.md`

2.05 rev2: other facts (derived)

- **Prop 10** is the bow image slot (pre-2.1 meaning), as in the existing prop table. No new meaning was found.
- **Swim state** is a client mode (10), not a prop. It is visible to others only through the animation.
- **Send helper** (also in 1.39): a message whose payload contains a newline is silently dropped, since the newline is the message terminator. The helper prefixes the id byte (id+32), adds the newline, queues, and flushes when the queue would pass 4095 bytes.
- **Disconnect on overload**: if more than 256 KiB (0x3ffff) of compressed output is pending (1.39), the client closes the link. The 2.05 figure was not checked.
- **Obfuscated attributes**: see `01-death-respawn.md`.
- **Respawn and CLAIMPKER** are explained in `01-death-respawn.md`.

`client-internals/decomp-findings-2026-10-24/06-first-graal.md`

1st Graal (Sept 1998 Delphi beta): protocol summary (derived)

Checked against `relay-1998.md` and `early-clients.md`.

Connection and login

- Default relay host the loopback address, port **14801**, through the event-driven socket component (matches the existing notes).

- No login token, no account, no password, no compression, no cipher. Lines are text ending in a newline; first character is the command.
- Join line: `N<game>#def`. The client always sends the name `def` and sends the real nickname later in a `D` payload.

Game (room) name: new detail

- The constant `Zelda2` is only used in one branch. In the other branch (selected by an argument given when the network client object is created) the client builds the room name from the current level file's base name: `Z` + base + `2`, for example a level `level1.zelda` would give a room `Zlevel12`. Right after that it runs a follow-up routine (not traced).
- Which branch is used for normal play was not traced. The two-client local mode (two client objects exist) is a likely user of the difference.
- The existing note says every 1st/2nd Graal join uses the constant `Zelda2`. That is true for one branch only.

D payload types received

The receive handler has cases for A B C D E F G H L M N O P R S T W (17 letters). That agrees with the existing note, which lists 16 sent types plus a receivable `D`. No other types.

Other confirmed facts

- Save files start with the marker `Graalsav1`.
- Level files use `.zelda` heavily and `.graal` in a few places; some built-in level names are tested by hard-coded comparisons (cave, house, village, level numbers).
- Web levels: the client has an HTTP check for "http" in a level path.

Not covered

Lobby line handling beyond I / + / - / D (P and C were not re-read), and feature behaviour (combat, items).

`client-internals/decomp-findings-2026-10-24/07-graal-139.md`

Graal 1.39 (Aug 2000 build): protocol summary (derived)

Checked against `versions-133-139.md`, `version-tokens.md` and the 1.39 entry in `v133-v139.toml`.

Login

- Host `<list-host-classic>`, port `14900` (strings in the binary, port parsed at connect time). No listserver step.
- Socket: TCP_NODELAY-style option set, non-blocking after connect.
- Login message is packet id 0 with payload: token `GNW07080` (built one character at a time), then one byte (account length + 32) and the account name, then one byte (password length + 32) and the password. Default account string is "guest". **Same as the existing note.**
- Level warp style join with id 0 and id 30 packets exist for offline `.code` weblevels in the join routine.

Framing and encryption

- Each message: id byte (id+32), payload, newline. A payload that contains a newline is silently dropped by the send helper (new detail, also true in 2.05).
- Queue is flushed when it would pass 4095 bytes. Each flush is zlib-compressed behind a 2-byte big-endian length. Sent to the socket in slices of 256 bytes.
- If more than 0x3ffff bytes (about 256 KiB) of compressed output are pending, the client closes the connection.
- No cipher and no type byte (generation 2). **Matches** the notes.

Packet ids

- Client to server ids passed as constants: 0 to 5, 7 to 24, 26 to 33, 80, 81. The existing list also has 6 (sent from a register-based site) and has no 25. **Matches.**
- Server to client dispatch handles ids 0 to 41, 43, 75, 100 (wire bytes 32 to 73, 75, 107, 132). **Matches** the existing `plo_known` list. Id 100 (raw data) and 75 are in the list.

Features seen in the binary

- DirectDraw and DirectSound through a Delphi DX component set, joystick and force-feedback classes, MNG and GIF animation, DDE classes (library code).
- Emoticon lookup by `emoticon_<letter>` file name with png/mng/gif extensions already exists (same routine as 2.05). Consistent with the existing "emoticons from 1.38".
- Sound files, shield/sword gif sets 1 to 4, `levels\webgifs\`, `levels\recordpos.txt`, `weblevels\`.

Death, hurt, sword and wall (filled 2026-10-31)

See `../../version-profiles/matrix/rules-139.md`. Death timer 0 → 30, STATUS dead bit and respawn key after counter > 27, hurt mode 40 ticks with five knockback steps, sword box 0–4 / <2, six-probe walk. Tag **[bin]**; hurt timings also **[lab]** in `../player-mannerisms-live.md`.

Not re-verified

The F7 popup items for 1.39 (menu strings are built character by character).

`client-internals/decomp-findings-2026-10-24/08-contradictions.md`

New facts and contradictions against Graal_analysis and the laundry notes

Contradicts or corrects

1. **Emote lookup (laundry 11-emotes-ganis.md step 4)** assumed Ctrl+letter maps to named files and sends PLI_SHOWIMG (24). In 2.05 the key path is local only (no packet). The lookup is a *prefix* match on `emoticon_<letter>` in the file list. SHOWIMG (24) is sent only by the script command `showimg`.
2. **1st Graal room name** is not always the constant `Zelda2`; one branch builds `Z` + level base + `2` (relay-1998.md says 1st and 2nd use `Zelda2`).
3. **TPersonAction** is the NPC "Set Npc's Action" script editor, not a player action menu.

New (not in existing notes)

- Death sequence timings (timer cap 30, input lock until 28, dead status bit from 28), respawn hearts by AP, respawn destination rule, no respawn packet.
- Wire sprite table (step to sprite) and the gani name to step rules; no swim sprite on the wire.
- Status-bit meanings (prop 18) and mode values.
- Sword hit evaluated on the victim; reach 0 to 4 tiles, lateral under 2, AP 100 immune.
- F7 popup item order for 2.05 and PM recipient encoding.
- Send helper drops payloads that contain a newline (1.39 and 2.05).

Confirmed (no change)

- 1.39 token, host and port, login layout, no cipher, framing, send id list, receive id list.
- 1st Graal port 14801, `NZelda2#def`, `Graalsav1`, D type set.

Labels

The upload labels for the three bundles do not match their contents (the 2.05 build was not in the folder labelled for it). File names were used to decide.

`client-internals/decomp-findings-2026-10-24-b/README.md`

Decompile findings, batch b (2.00, 2.17.1, 2.22, 2.31 + Drive clients), 2026-10-24

Derived facts only. **See also batch c (`../decomp-findings-2026-10-24-c/`): 20 builds from 2.00 to 2.1 test, NewWorld and NewMain; it supersedes the 2.00 and 2.05 parts of this batch where they differ.** No code, addresses or decompiler names. Lab-only runs (loopback network, private display). Built from decompiles of 2.00, 2.17.1, 2.22 and 2.31, plus the Drive clients set and lab captures.

File	Content
01-tamper-check-id39.md	how the 10 bytes of client PLI 39 are built; per-build words; when sent
02-server-to-client-layouts.md	payloads for server ids above 48 and for 25, 39, 41, 44, 47, 156, 168-189; 2.17.1 vs 2.22 vs 2.31
03-periodic-ids-2.31.md	2.31 timer: PLI 39 every 3.2 s, PLI 24, PLI 46
04-v2.00-client.md	2.00 login, id tables, token allow-list, lab setup
05-open-items.md	what is settled and what is still open
06-crosscheck.md	new facts, confirmations and corrections against the repo
07-lab-findings-46-186-200.md	lab: PLI 46 on every level entry, 186 trailer behaviour, 2.00 listserver handshake (observed vs synthetic)
08-lab-findings-levelchange-24-178.md	lab batch c: PLI 46 on doors, links and server warps (2.31 yes, 2.22 never), PLI 24 relay, PLO 178 field order and reconnect (observed vs synthetic)
../drive-clients-inventory.md	what the Drive clients set contains; Wine status

Tables (all under `version-profiles/`):

File	Content
plo-layouts-217-231.tsv	one row per server id: fields, version gates, confidence
plo-handled-ids-by-build.tsv	handled server-to-client ids per build, 2.00 to 2.31
pli-sent-ids-by-build-20x.tsv	client-to-server ids per 2.0x build

File	Content
tampercheck-words.tsv	CRC words and lab bytes per build (companion to `version-profiles/tampercheck-values.tsv`)
version-tokens-by-build.tsv	login tokens per build
lab-enterlevel-231.tsv	PLI 46 sends of one 2.31 gmap run
lab-boardmodify2-186-trailer.tsv	`PLO_BOARDMODIFY2` trailer and length test cases (synthetic frames)
lab-200-listserver-handshake.tsv	2.00 listserver and gserver handshake with provenance
lab-levelchange-46-222-231.tsv	PLI 46 and PLI 39 on 18 level changes, 2.31 and 2.22
lab-showimg24-relay.tsv	PLI 24 payloads and the server's relay as PLO 32
lab-serverwarp178.tsv	PLO 178 real chain and field-order tests
drive-clients-exes.tsv	every exe in the Drive clients set (folder, size, PE file version)

`client-internals/decomp-findings-2026-10-24-b/01-tamper-check-id39.md`

Tamper-check (client PLI 39): how the 10 bytes are built

Derived facts only. All six lab values (two captures each of 2.17.1, 2.22 and 2.31) were reproduced offline from the client executables, so the description below is verified, not guessed. Lab captures and re-send timings: `version-profiles/tampercheck-values.tsv`.

What the 10 bytes are

- Two 32-bit words, A then B, each written as 5 bytes. The 5-byte form is NOT the normal GINT5 and NOT masked:
  - byte 1 = ((v >> 28) & 0x0F) + 0x20
  - byte 2 = ((v >> 21) + 0x20) & 0xFF
  - byte 3 = ((v >> 14) + 0x20) & 0xFF
  - byte 4 = ((v >> 7) + 0x20) & 0xFF
  - byte 5 = (v & 0x7F) + 0x20 Only bytes 1 and 5 are range-limited, so bytes 2-4 can be below 0x20 or above 0x7F. That is why the lab saw bytes under 0x20 and why "two GINT5" is wrong. (Layout is the same in 2.17.1, 2.22, 2.31.)
- Both words are standard reflected CRC-32 values (polynomial 0xEDB88320, initial value 0xFFFFFFFF) computed over the client's OWN memory image (the running exe, in memory), chained: the first call finishes with the usual final inversion, its result is used as the starting value of the next call, and the last call of each word skips the final inversion.
- Word A covers 8-9 contiguous ranges of the code section (a sliding selection of code, not the whole section).
- Word B covers one large code range plus one block of static data (a table of string pointers in the data section).
- Nothing in the input depends on login key, account, level or time. So per build the result is a constant, exactly as the lab saw. The value only changes if the exe bytes change (a patched or re-packed exe changes it; an unpatched original does not).

Per-build values (words in hex, lab bytes in hex)

See `version-profiles/tampercheck-words.tsv`. Short form: - 2.17.1: A ab637cbe, B 3e48dc0c, bytes 2a7bad195e231243d82c - 2.22: A 5dcb77f6, B bc736311, bytes 250e4d0f962b03ede631 - 2.31: A 9ecd4bc0, B ecea65e2, bytes 291655b7602e87c9eb82

What a Seihai client must do

- Replaying the constant per emulated build is enough (it is what the real client sends). Computing it needs the original exe bytes plus the range lists, which are inside each exe; not needed unless Seihai wants to accept patched exes.
- Position/timing per build: see `03-periodic-ids-2.31.md` and the table at the end of this file.

When it is sent

build	login	later
2.17.1	yes, once	never in ~90 s of lab
2.22	yes	re-sent by the level-change routine on every level change (so each gmap edge crossing in the lab); no timer
2.31	yes	every 3.2 s; a 50 ms main-form timer drives it (see 03)

Does the server verify it?

Lab: GServer-v2 accepts all three and logs nothing. The decompiles only show the sender; no check on the receive side of the client. Treat as sent-and-ignored for a compatible server.

`client-internals/decomp-findings-2026-10-24-b/02-server-to-client-layouts.md`

Server -> client payloads above id 48 (and 25, 39, 41, 44, 47, 156, 168-189): 2.17.1, 2.22, 2.31

Source: the receive dispatchers of the 2.17.1, 2.22 and 2.31 clients, read from the disassembly. Full table: `version-profiles/plo-layouts-217-231.tsv`. Handled-id set per build: `version-profiles/plo-handled-ids-by-build.tsv`.

Dispatcher shape (all builds)

- id = first byte - 0x20; bound check; byte-index table; dword jump table; every unhandled id shares one default arm that does nothing.
- Dispatch bounds: 2.17.1 up to id 185 (bound 0xb9); 2.22 up to 186 (0xba); 2.31 up to 189 (0xbd).
- Number encodings read by the handlers: GCHAR = byte-0x20; GSHORT = 7-bit shifts (b0<<7 + b1); GINT3 shifts 14,7,0; GINT4 21,14,7,0; GINT5 28,21,14,7,0. Text payloads are the rest of the line. Several "comma text" ids go through the TStrings comma-text parser (double quotes allowed).

Handled ids above 48

- 2.17.1: 49, 68, 69, 75, 100, 150, 151, 153-156, 165, 166, 168-185. (79 handled ids in total.)
- 2.22: same + 186. (80 total.) Id 7 is a three-instruction stub forwarding to a helper (in 2.17.1 the body is inline); wire format of 7 unchanged as far as the diff shows.
- 2.31: the 2.22 set + 84 + 189, minus 44. (81 total.)
- Ids 50-67, 70-74, 76-83, 85-99, 101-149, 152, 157-164, 167 are unhandled in all three (default arm). 152 and 167 are NOT handled in any of them.

Layouts, short

(see `version-profiles/plo-layouts-217-231.tsv` for the full text and version gates; "dec" = decoded from the binary) - 25 SIGNATURE: GCHAR v. v=1 quits; v=0 quits unless host is the loopback address; other values ignored. - 39 LEVELMODTIME: raw rest passed to active gmap/level object (needs an active level). - 41 STARTMESSAGE: raw RTF/HTML text, cached as servermessage.rtf/.html, shown in viewer. - 44 HASNPCSERVER: no payload, 2.17/2.22 only, gone in 2.31. - 47 STAFFGUILDS: comma list. - 49 PLAYERWARP2: GCHAR x2, GCHAR y2, GCHAR z (unused here), GCHAR gmapX, GCHAR gmapY, level name. x,y halve (value/2.0). gmapX/Y only used for .gmap levels, clamped to the grid. - 68/69 LARGEFILESTART/END: raw name; 84 LARGEFILESIZE (2.31 only): GINT5. - 75 PROFILE: repeated GCHAR n + n chars; first field = account, rest = lines. - 100 RAWDATA: GINT3 length. - 150 NPCDEL2: GCHAR n, n-char level name, GINT3 npc id. 151 HIDENPCS: none (removes NPCs 1..999 of active level). - 153 SAY2: text. 154/155 FREEZE/UNFREEZE: none. 156 SETACTIVELEVEL: name, .gmap handled specially; 2.31 adds a follow-up step. - 165 MOVE: GINT3 id + rest. 166 SHOWIMGNPC: GINT3 id + props. 189 MOVE2 (2.31): same shape as 165, extra flag. - 168: payload ignored (local reset). 169: URL text. 170 GCHAR flag. 171/172: comma text txt,png,x,y (needs >=4 fields). 173 text. 174 GCHAR flag. - 175 SHOOT: GSHORT player id + rest, only when in the active level. - 176/177: mode switches, no payload. 178 SERVERWARP: comma text with >=4 fields, field 0 unused, 1 = name, 2 = host, 3 = port (lab-confirmed on 2.31, `08-lab-findings-levelchange-24-178.md`). - 179 RPGWINDOW: text. 180 STATUSLIST: text. 181: GSHORT id + GCHAR toggle. - 182 LISTPROCESSES: no payload; client replies PLI 44 with its process list. 183 HASPROCESSRUNNING: names; client replies PLI 45 = GCHAR 0/1 (whether the server's string occurs in its own process list). (These are PLI ids sent BY the client; the server-side PLO 44 is something else.) - 184 TAKESCREENSHOT: file name, only ".png" accepted, saved under a screenshots folder. - 185 BOARDHEIGHTS: GCHAR mapX, mapY, blockX, blockY, w-1, h-1 then w*h cells of two raw bytes, value = (b0-0x20-50) + (b1-0x20)/128.0. - 186 BOARDMODIFY2 (2.22+): GCHAR mapX, mapY, x, y, w, h then w*h GSHORT tiles row-major (bytes after the tiles are ignored padding: lab-tested with injected frames, see `07-lab-findings-46-186-200.md`).

Differences 2.17.1 -> 2.22 -> 2.31 that change behaviour or ids

- 2.17.1 -> 2.22: new id 186; id 7 refactored into a helper; every other handler's wire format identical after diffing 37, 42, 49, 75, 150, 151 (only local-variable/offset noise). Tamper check re-sent on level change (new). Login token changes GNW22122 -> GNW03014.
- 2.22 -> 2.31: id 44 no longer handled; new ids 84 and 189; id 6 (level name) now also compares with the current level name; 68 clears the stored file size; 156 gets an extra step; object offsets shift by 0x40 in the player/level classes (irrelevant on the wire). Timer-driven PLI 39/24 and PLI 46 added (see 03).

`client-internals/decomp-findings-2026-10-24-b/03-periodic-ids-2.31.md`

2.31 periodic traffic: PLI 24, PLI 39 re-send, PLI 46

Mapping of the lab observations to the client's behaviour (all values are durations).

PLI 39 re-send every ~3.2 s

- The main form of 2.31 has a 50 ms timer. Its handler keeps a counter modulo 64 (64 = length of the 64-character alphabet string used there), so one full cycle = 64 x 50 ms = 3.2 s.

- Phases of the cycle (counter value -> action):
  - 1: anti-debug probe (a privileged port-I/O instruction guarded by an exception handler, then a breakpoint-trap check), and a 5000 ms timer is armed.
  - 10: recompute CRC word A of the tamper check.
  - 20: scan all top-level windows (title and class) against an obfuscated blacklist (cheat-tool detection). The list was not decoded and is not needed.
  - 30: recompute CRC word B.
  - 31: send PLI 39 with the freshly computed A+B.
- Because A and B recompute to the same constant each time (see 01), the 10 bytes are identical on every re-send (lab confirmed). I did not find the blacklist scan or the probe result feeding into the words; what the client does on a hit was not traced.
- 2.17.1 and 2.22 have no such timer. 2.22 only re-sends from the level-change routine.

PLI 24 about every 5.1 s (1 byte, lab value 9)

- 2.31 only. The 5.1 s cadence fits the 5000 ms timer armed in phase 1 (5.0 s plus timer granularity). That link is an inference, not traced to the end.
- **Lab batch c (`08-lab-findings-levelchange-24-178.md`):** 2.31 sends it at login and every 5.11 to 5.17 s (median 5.13 s) whatever it is doing, in every level; level changes neither trigger nor reset it. **2.22 also sends 24 = 9** every 5.1 s, but only while another player is in the same level (0 in 3 solo sessions), so "2.31 only" is wrong for two-player 2.22. The stock server relays any payload unchanged as `PLO_SHOWIMGPLAYER` (32) to the others in the level; injected payloads caused no reaction.
- The payload byte (value 9) is a constant in the sender; I could not tie it to anything else. Meaning NOT resolved (heartbeat-like by behaviour only). Treat as: send GCHAR 9 as PLI 24 every ~5 s on a 2.31 emulation; unknown whether servers care.

PLI 46 on every level entry (GCHAR 1)

- Constant 1 byte (`GCHAR 1`). **Updated by the lab run of batch b2:** it is sent on every level entry, not only at login (7 sends in one gmap run: login plus six edge crossings), always just before the props message that carries the cell prop (43 or 44) and the wrapped coordinate. Details: `07-lab-findings-46-186-200.md`.
- 2.31 does not re-send PLI 39 at a crossing. Not sent by 2.17/2.22.
- **Lab batch c:** also sent on doors, edge links and server-side warps (always just before the first props of the new level; in the same frame as a cached-level link request, otherwise in the first frame after the server's level answer). 2.31 does not re-send 39 there either. 2.22 sent no 46 in 0 of 8 transitions, but sent 39 after each client-initiated link request. Details: `08-lab-findings-levelchange-24-178.md`.

What to implement

- 2.31: send PLI 46 = GCHAR 1 at login and on each level change; PLI 39 (constant) every 3.2 s; PLI 24 = GCHAR 9 every ~5.1 s.
- 2.22: PLI 39 at login and after each client-initiated level-link request (not after a server-side warp); no 46; PLI 24 = 9 every ~5.1 s while another player is in the level. 2.17: login only.

`client-internals/decomp-findings-2026-10-24-b/04-v2.00-client.md`

Graal 2.00 (and rev1-3, 2.01-2.05): login, tables, lab setup

> **Batch c (`../decomp-findings-2026-10-24-c/`) supersedes the 2.00 and 2.05 parts of this note where they differ.** It decompiled all four 2.00 builds and 2.01 to 2.1 test with a rebuilt dispatcher. The tokens, the id sets and the dropped id 26 here match it. Corrections: id 6 is a real sender (below), and the four 2.00 builds have identical handler shapes.

Sources: the 2.00 decompile and the four 2.00 builds from the Drive clients set (`../drive-clients-inventory.md`). All four 2.00 builds give the same token and the same tables. Derived facts only.

Login string (gserver)

- Built from: GCHAR(random key & 0x7F) as the key byte, then the 8-char token `GNW24011` (assembled character by character from immediates, so a plain text search finds nothing), then GCHAR(len account) + account, GCHAR(len password) + password. It is sent through the client's send method with packet type 0 (same "type 0, key byte, token, account, password" shape as 2.05 and 2.17). Account and password are NOT hidden or hashed in 2.00.
- The key byte seeds the gen-3 stream cipher framing used afterwards (zlib + one junk byte per message after the first, as documented for 2.05 and 2.17). No new framing in 2.00.
- Tokens for the whole 2.00-2.1 family: `version-profiles/version-tokens-by-build.tsv` (all confirmed from the binaries) and `../version-tokens.md`.

Server -> client ids handled (2.00)

- Dispatch bound 0x9b; 53 ids: 0-45 (including both 25 and 26), 75, 100, 150, 151, 153, 154, 155. No 46-49, 156, 165-169, 68-69.
- Growth: 2.01 +46, +156 (55 ids); 2.02 +47 (56); 2.03 +48, +165 (58); 2.04 drops 26 (25 stays in every build; 2.04 does not swap one for the other) and adds 166; 2.05 +168 (59); the 2.1 test builds +49, +169 (61). Full list: `version-profiles/plo-handled-ids-by-build.tsv`.
- `PLO_SIGNATURE` (25) is handled in 2.00 already, so the kill-switch exists from the first 2.x client.
- `PLO_NPCACTION` (26) is handled by 2.00-2.03 and dropped in 2.04. Later clients ignore it. Its payload is not decoded here.

Client -> server ids the 2.00 binary can send

0-24, 26-30, 32-34, 80, 81. Compared with 2.05 rev2 (0-24, 26-30, 32-38, 80, 81): 2.00 lacks 35-38; 35 and 36 arrive in 2.01, 37 in 2.02 rev2, 38 in 2.03. Table: `version-profiles/pli-sent-ids-by-build-20x.tsv`. Method: call sites of the client's send method with a constant id. The 2.05 rev2 result matches the id set already in `protocol/packet-ids.md`, so the method is sound. Id 6 (`PLI_TOALL`, chat) shows up in every build. **Correction (batch c):** it is a real sender in all 20 builds, with chat text clipped to 200 characters, not a scan artefact (`../decomp-findings-2026-10-24-c/05-pli-client-to-server.md`).

Token allow-list

- A GServer-v2 gserver has a text allow-list of tokens. For a lab, a **config entry** for `GNW24011` in that list is enough. No binary patch is needed on the server, and none is needed in the client for the login itself.

(Earlier notes, `version-profiles/version-coverage-2026-10-24.md`, said an allow-list patch; that meant this config entry.)
- Still needed by the 2.00 client: a list server it can reach. The list host is DES-hidden inside the exe (scheme in `../des-and-hidden-strings.md`, port 14921 per `protocol/listserver.md`), and 2.00 does not read `arrays2.dat`. For a lab, resolve that name to the lab listserver and run the lab listserver on 14921, or patch the hidden string.
- **Partly observed (batch b2, `07-lab-findings-46-186-200.md`):** the 2.00 client sends one plain zlib frame, `GCHAR 0` + `newmain` + LF, with no account or password, then makes no second listserver connection and goes straight to the gserver. 2.00 accepted a 2.17-style two-frame list reply, but that reply was **synthetic** (a lab-only forwarder translated the request). What a real 2.00-era listserver sends is still unobserved. Table: `version-profiles/lab-200-listserver-handshake.tsv`.

Does it run?

Yes under Wine 10 (separate prefix, loopback-only lab network, private display), with a data-file caveat. The Drive 2.00 exe alone stops with "Cannot open file commands.rtf", because the Drive folder holds only the exe. Placing the data files of a NewMain 26 folder (commands.rtf, images, languages) beside it makes it reach its start screen. A copy of the 2.22 data folder also works for the start screen and login, except that `objects.dat` is missing there and 2.00 needs it (nickname box, Training / Options / Start). See `../drive-clients-inventory.md`.

`client-internals/decomp-findings-2026-10-24-b/05-open-items.md`

Open items after the 2.00 / 2.17.1 / 2.22 / 2.31 decompile pass

Settled by this pass (for reference)

- **`PLO_HIDENPCS` (151)** removes the NPCs with ids 1 to 999 of the active level. Id 0 is excluded. (Earlier note: "probably ids below 1000".)
- **`PLO_SIGNATURE` (25)** value 1 makes the client quit. Value 0 makes it quit unless the connected host string is the loopback address. Any other value is ignored. Identical in 2.17.1, 2.22 and 2.31, and the id is handled from 2.00 on. The value 73 that public servers send at login ("more than eight players") is in the ignored group, so it never triggers the quit rule.
- **Tamper check:** how the 10 bytes are built (`01-tamper-check-id39.md`), and why 2.31 repeats it every 3.2 s (`03-periodic-ids-2.31.md`).
- **2.00 login** and its allow-list requirement (`04-v2.00-client.md`).
- **`PLI_ENTERLEVEL` (46)** (lab, batch b2): `GCHAR 1`, sent on every level entry, just before the props message with the cell prop (43/44) and wrapped coordinate. 2.31 does not re-send the tamper check at a crossing. **Batch c:** also sent on doors, edge links and server warps (2.31); 2.22 never sends it. Only a script-initiated warp is untested (`08-lab-findings-levelchange-24-178.md`).
- **`PLO_BOARDMODIFY2` (186) trailer** (lab, batch b2, synthetic frames): bytes after the tiles are ignored; a second record there is not applied; one byte short of `6 + 2*w*h` is rejected silently; a block past column 63 is clipped. See `07-lab-findings-46-186-200.md`.

Still open

Item	State	What would settle it
Client `PLI_SHOWIMGPLAYER` (24), 1-byte payload value 9 (2.31 always; 2.22 only while another player is in the level; about every 5.1 s)	**Partly settled by lab batch c:** the stock server relays any payload unchanged as 32 and nothing reacts (injected payloads, synthetic); 2.22 sends it too in a two-player level. The 5.1 s cadence fits the 5000 ms timer armed in the 2.31 main-form timer (inference, not traced to the end). The 9 is a constant in the sender and is not tied to anything else. Meaning unknown.	Trace the sender's caller chain; a real client's reaction to a 1-byte or emote-form 32 with real image files; the exact 2.22 send condition
`PLO_SERVERWARP` (178) field meanings	**Settled by lab batch c (2.31, synthetic 178 frames plus one real chain):** field 0 unused, field 1 = server name (not needed to connect), field 2 = host, field 3 = port; the client closes the connection and reconnects with a fresh login, no listserver step; fewer than 4 fields is ignored. Still open: whether field 1 is used elsewhere; 2.22; host names, port range, extra fields; the text of a real service (the lab text came from the open-source listserver reimplementation).	A capture of a real serverwarp answer
`PLO_SETACTIVELEVEL` (156), extra step in 2.31	2.31 adds a follow-up step (a call with all-zero arguments, then another call) that 2.17.1 and 2.22 lack. Purpose not resolved. No wire change.	A longer read of the 2.31 handler
Client to server ids 36, 42 and 45 in 2.22	Not matched to a call site. This pass scanned 2.0x only, not 2.17/2.22/2.31, for client senders. In 2.0x, 36 is sent from 2.01 on; 42 and 45 never appear in any 2.0x or 2.1 test build. 45 is the reply to `PLO_HASPROCESSRUNNING` (183) per the 2.22 and 2.31 receive side.	The same send-site scan over the 2.17.1, 2.22 and 2.31 decompiles
2.00 listserver login frames	**Partly observed.** Client side seen: one plain zlib frame `GCHAR 0` + `newmain`, no account or password, no second listserver connection; the list host is DES-hidden and was redirected by name. The reply 2.00 accepted was **synthetic** (forwarder-injected 2.17-style request, lab listserver reply). A real 2.00-era listserver reply is unobserved (`07-lab-findings-46-186-200.md`).	A capture of a real 2.00-era listserver reply, or a decompile read of what 2.00 sends in later list frames
2.31 window-title blacklist (cheat-tool scan inside the 50 ms timer)	Deliberately not decoded. What the client does on a hit is not traced, and the scan result was not found feeding the tamper words.	Not needed unless a server is seen to enforce the tamper check
Per-build handled-id sets for 2.01 to 2.04 and the 2.1 test builds	**Settled by batch c** (decompiled; dispatcher rebuilt; 20 builds): see `../decomp-findings-2026-10-24-c/04-plo-server-to-client.md`. Field layouts of the long handlers are still not extracted.	Per-handler manual reading
`PLO_NPCACTION` (26) in 2.00 to 2.03	Batch c read the start of it: `GINT3` npc id first, acts only when the player's level is the active level. The rest of the payload is not decoded (`../decomp-findings-2026-10-24-c/04-plo-server-to-client.md`).	A longer read of the 2.00 to 2.03 handler
2.1 to 2.16 and 2.3	No binaries anywhere (`../drive-clients-inventory.md`).	New binaries

`client-internals/decomp-findings-2026-10-24-b/06-crosscheck.md`

Cross-check against the repo (after PRs #30 and #31)

New facts (not in the repo before this pass, or only marked unverified)

- Tamper-check derivation: two chained CRC-32 words over the client's own memory, 5-byte non-GINT5 encoding with unmasked middle bytes. All six lab values reproduced. Words per build: `version-profiles/tampercheck-words.tsv`.
- 2.31 timer: a counter modulo 64 on a 50 ms timer gives 3.2 s; phases 1, 10, 20, 30, 31 (probe, CRC A, window scan, CRC B, send). The 3.2 s is derived, not only observed.
- Payload layouts for every server id above 48 plus 25, 39, 41, 47, 156, 165, 166, 168 to 189 that the repo had as names or blank cells: `version-profiles/plo-layouts-217-231.tsv`. Notable: 49 (x and y halved, z unused, gmap cell, then level), 75 (length-prefixed fields), 150, 171 and 172 (comma text, at least 4 fields), 178 (at least 4 comma fields, 1 to 3 used), 181, 182 and 183 reply ids (client 44 and 45), 184 (only `.png`), 185 (height cell formula), 186 (GSHORT tiles).
- `PLO_SIGNATURE` (25) semantics: value 1 quits; value 0 quits unless the host is the loopback address.
- Login tokens for the whole 2.0x family are confirmed from the binaries and all match the repo (`version-profiles/version-tokens-by-build.tsv`).
- Client to server id sets per 2.0x build (`version-profiles/pli-sent-ids-by-build-20x.tsv`): 2.00 lacks 35 to 38; 35 and 36 arrive in 2.01, 37 in 2.02 rev2, 38 in 2.03.
- 2.31 `PLO_LEVELNAME` (6) now also compares against the current level; 68 clears the stored file size; 156 has an extra step.
- 2.00: login string shape with plain account and password, no `arrays2.dat`, and runs under Wine once the data files are supplied.
- `PLO_NPCACTION` (26) is handled by 2.00 to 2.03 only.

Confirmations

- Handled-id set sizes (80 for 2.22, 81 for 2.31; 2.31 drops 44, adds 84 and 189) match `version-profiles/2.22-2.31.md` and `protocol/client-versions.md`.
- The 2.1 test builds send the 2.05 token, as `client-internals/version-id-map.md` says.
- The `protocol/packet-ids.md` names for 168 to 189 match the 2.22 and 2.31 behaviour (ghost, minimap, kingdoms, new world mode, serverwarp, RPG window, status list, process list, screenshot, board heights, board modify 2).
- 2.31 dropping 44 is consistent with the `protocol/packet-ids.md` note "client 5.07+ ignores it".
- The 10-raw-bytes description of PLI 39 (PR #30) is explained by the unmasked encoding.

Corrections made in this change

- `PLO_HIDENPCS` (151): was "probably removes NPCs with id < 1000"; decoded as ids 1 to 999 of the active level.
- `version-profiles/version-coverage-2026-10-24.md` and `client-internals/version-profiles/141-217.toml` say a lab needs an allow-list "patch" for 2.00. A plain allow-list config entry is enough (`04-v2.00-client.md`). The report now carries a status update; the TOML note is unchanged (it only says stock GServer-v2 treats the token as unknown, which is still true).
- `version-profiles/2.22-2.31.md` lists client to server ids 36, 42 and 45 as unmatched; that is still true (this pass did not scan 2.17/2.22/2.31 send sites). Listed in `05-open-items.md`.

Corrected by batch c

- Id 6 (`PLI_TOALL`) is a real sender in every 2.0x build, not a possible scan artefact (`04-v2.00-client.md` corrected).
- 2.04 only drops id 26; id 25 is handled in every build. Nothing is swapped.
- The 2.00 builds (r0 to r3) share handler shapes and one token. Batch c supersedes the 2.00 and 2.05 statements here where they differ: `../decomp-findings-2026-10-24-c/10-open-questions-and-corrections.md`.

`client-internals/decomp-findings-2026-10-24-b/07-lab-findings-46-186-200.md`

Lab findings: PLI 46, PLO 186 trailer, 2.00 listserver handshake (batch b2, 2026-10-24)

Derived facts only. Lab-only runs (loopback network, private display, placeholder accounts). No public server was contacted. Tables are under `version-profiles/`:

Table	Content
`lab-enterlevel-231.tsv`	the seven PLI 46 sends of one 2.31 run, with the props that follow
`lab-boardmodify2-186-trailer.tsv`	the six `PLO_BOARDMODIFY2` test cases on 2.22 and 2.31
`lab-200-listserver-handshake.tsv`	the 2.00 listserver and gserver handshake, step by step, with provenance

How to read the provenance

- **Observed** means a real client talking to the stock lab servers, with nothing in between that changed the traffic.
- **Synthetic** means the lab needed a lab-only helper in the path. Two results rest on one:
  - The **2.00 listserver reply** was not sent by a real listserver. The lab listserver only answers a 2.17-style request, so a forwarder added such a request upstream after the client's frame. What 2.00 accepts is confirmed; what a real 2.00-era listserver sends is **unobserved**.
  - The **id 186 frames** were injected by a re-encoding proxy. Stock GServer-v2 defines the id only as an unknown packet and never sends it, and its scripting has no raw-packet send. So the client reactions below are real, but no server of any kind has been seen sending 186.

PLI_ENTERLEVEL (46), real 2.31 client [observed]

- Payload is always one byte, `GCHAR 1`. It is **not** a login one-shot: it is sent on every level entry. One run produced 7 sends: login plus six gmap edge crossings (east, west, east, west, south, north). Table: `version-profiles/lab-enterlevel-231.tsv`.
- Order inside the client's frame: 46 first, then the props message that carries the cell prop (43 for a horizontal crossing, 44 for a vertical one) and the wrapped local coordinate. At login it is the first message of the second client frame, before PLI 35 (adjacent levels) and PLI 44.
- It is client-initiated and does not wait for the server. The server's answer to a crossing is the same as for 2.22 (level name, mod time, board, gmap name, 174, 10, 42, 156).
- **2.31 does not re-send the tamper check (id 39) at a crossing.** Its 39 stays on the 3.2 s timer, and 24 stays about every 5.1 s. 2.22 is the build that re-sends 39 at a crossing.
- Not tested: a level change that is not a gmap edge crossing (door, warp). The "every level entry" wording rests on the gmap crossings plus login.
- Not sent by 2.17 or 2.22 (client id sets in `version-profiles/2.22-2.31.md`).

- Emulation rule for a 2.31 profile: send PLI 46 = `GCHAR 1` at login and on each level change, just before the props message that changes the cell prop.

PLO_BOARDMODIFY2 (186), 2.22 and 2.31 [client reaction observed, frames synthetic]

Layout: `GCHAR mapX, mapY, x, y, w, h`, then `w*h` GSHORT tiles. Results, identical on 2.22 and 2.31 (`version-profiles/lab-boardmodify2-186-trailer.tsv`): - The handler's length check is "at least `6 + 2*w*h`". Bytes after the tiles are **unread padding**. - A trailer of arbitrary bytes has no effect and does not disconnect. - A complete second record placed in the trailer is **not applied**. - One byte short of `6 + 2*w*h` is rejected silently: nothing is drawn and the client stays connected. A payload that is much shorter behaves the same. - A block that runs past board column 63 is **clipped**, not rejected: only the columns inside the board are drawn. - Not tested: `mapX`/`mapY` other than the current cell, or out of range.

2.00 listserver handshake [client side observed, list reply synthetic]

Setup: the 2.00 exe run beside a copy of the 2.22 data folder, plus `objects.dat`, which 2.00 needs and the 2.22 folder does not contain (it came from a NewMain 26 folder). The hidden list host resolved through a per-namespace hosts file, so the exe was not patched. Table: `version-profiles/lab-200-listserver-handshake.tsv`. - **Observed:** the client sends **one** plain zlib frame: `GCHAR 0` + `newmain` + LF, then half-closes. No account, password, token or key. This differs from 2.17, whose first frame also carries `GCHAR 1 account password`. - **Synthetic:** the list reply (two plain zlib frames: id 0 list; ids 2, 3, 5) came from the lab listserver after a forwarder injected a 2.17-style request. 2.00 parsed and drew it, so the 2.17-style reply format is **accepted** by 2.00. What a real 2.00-era listserver replies to the bare `newmain` request is **still unobserved**. - **Observed:** pressing Connect opens no second listserver connection. The client goes straight to the gserver with type 0, key byte, `GNW24011`, `GCHAR len + account`, `GCHAR len + password`, then gen-3 framing (junk-byte rule, 0 mismatches). The lab server's allowed-versions entry for `GNW24011` was enough; no patch. - **Observed:** first client ids after login: 34, 23, 18 (twice, the gr.x and gr.y flags), 2. None of 37, 39, 46, 24 or 44 appear. The server's 168, 190, 194, 47 and 182 are ignored without a disconnect. - **Data-file caveat:** the 2.22 data folder alone is not enough for 2.00; `objects.dat` is missing there. Without it 2.00 opens a blank grey window after Connect and never dials (`../v2.00-connect-path.md`).

`client-internals/decomp-findings-2026-10-24-b/08-lab-findings-levelchange-24-178.md`

Lab findings: PLI 46 on level changes, PLI 24, PLO 178 (batch c, 2026-10-24)

Derived facts only. Lab-only runs: real 2.22 and 2.31 clients against the stock open-source game server and listserver, on a loopback-only lab network, with placeholder accounts. No public server was contacted. Follows `07-lab-findings-46-186-200.md` (batch b2). Tables are under `version-profiles/`:

Table	Content
`lab-levelchange-46-222-231.tsv`	18 level changes (door or edge link, server-side warp) on 2.31 and 2.22: whether PLI 46 and PLI 39 went out, and where
`lab-showimg24-relay.tsv`	PLI 24 payloads (the real 9 and eight injected ones) and how the server relayed them
`lab-serverwarp178.tsv`	PLO 178 tests: the real chain and five injected field-order cases

How to read the provenance

- **Observed**: a real client and a real lab server, nothing altered.
- **Synthetic**: a lab-only re-encoding proxy rewrote or injected a message. The client's or server's reaction is what was observed. It says nothing about what a real public service sends.
- **Unresolved**: not answered by this run.
- In the 178 table, `<name>`, `<lab host>` and `<port>` are placeholders for the lab server's name, a loopback address and its port.

A. PLI_ENTERLEVEL (46) on doors, edge links and server warps

Table: `version-profiles/lab-levelchange-46-222-231.tsv`. 2.31: 2 sessions, 10 transitions. 2.22: 1 session, 8 transitions. Both mix first visits and cached visits.

2.31 [observed] - Sends `46 = GCHAR 1` on **every** level entry, not only on a gmap crossing: a level link (a door or an edge strip) and a server-side warp (the chat command `warpto`). The payload never changes. - The position depends on who starts the change and on whether the client has the level cached: - **Level link, level not cached.** The client sends `PLI_LEVELWARP` (0) with x, y and the level name. The server answers (ids 14, 6, 100, 101, 39, 1, 0, 174, 10, 42, 156). Then the client sends one frame: `46`, then the props. So 46 comes **after** the server's answer. - **Level link, level cached.** The client sends `PLI_LEVELWARPMOD` (30), which adds the level's mod time before x, y and the name. The 46 is in the **same frame** as the 30, right after it and before the props, so before the server's answer. That answer has no id 14, and no 100/101 when the board is cached too. - **Server-side warp.** The server sends 14, 6 and the level data. The client then replies with a frame whose first message is 46. The props follow in the same frame only if the position changed; a warp to the same x, y gives `46` alone. - **Gmap crossing** (batch b2): 46, then the props, in one frame before the server's answer. - One rule covers all four: 46 goes out when the client has the level and is about to send the first props of it. At login it is in the login frame after the `PLI 35` messages and before `PLI 44` and the props. - **2.31 does not re-send the tamper check (39) on a door, a link or a warp.** Its 39 stays on the 3.2 s timer.

2.22 [observed] - **Never sends 46**: 0 of 8 transitions, and not at login. - It does send the tamper check (39, the same 10 bytes) in the same frame as every **client-initiated** level-link request (`0` or `30`, then `39`), as it does at a gmap crossing. It does not send 39 after a server-side warp.

Not done / unresolved - A level change started by a level NPC script (`setlevel2`). The lab's level scripts never ran, so there is no capture of a script warp. Whether it looks like a link on the wire is unresolved. - Separate lab observation, not investigated: an NPC whose image was a large sprite sheet made the 2.31 client stop sending anything (no heartbeat) for over a minute.

B. PLI_SHOWIMGPLAYER (24)

Table: `version-profiles/lab-showimg24-relay.tsv`.

What the real clients send [observed] - **2.31** sends `24 = GCHAR 9` in its first frame after login (after 46 and 44, with the props) and again every 5.11 to 5.17 s (median 5.13 s; one gap of 5.5 s), idle or walking, in every level used. 48 messages in two solo sessions. The value never changed. A level change neither triggers nor resets it. - **2.22 also sends `24 = 9`**, at login and then every 5.1 s, **but only while another player is in the same level**: 0 messages in 3 solo sessions of 70 to 100 s each. Earlier notes calling 24 "2.31 only" are wrong for 2.22 with two players. The exact condition (another player present, their props received, or the same level) was not isolated. **Unresolved.**

Server reaction [observed in the source and on the wire] - The stock server relays 24 unchanged as `PLO_SHOWIMGPLAYER` (32) = `GSHORT sender id` + the client's payload, to every other player in the level area (not back to the sender). Both real clients were seen receiving the other's 9 every 5 s.

Injected payloads [synthetic] - A lab-only proxy replaced the payload of each client 24 with `GCHAR 0`, `1`, `5`, `14`, `58`, `95`, the six bytes `xx.png`, and an empty payload. The server relayed each one verbatim as 32. No disconnect, no invalid-packet entry in the server log, no error. Neither 2.31 nor 2.22 showed any visible reaction: screenshots matched the baseline apart from an animated status bar. The lab had no player sprites or emoticon files, so a drawn image could not have shown anyway.

Conclusion - The stock server treats 24 as an opaque payload to relay. The 2.31 form is a single byte 9, unlike the four-field emote form (index, x, y, name length and name) that the 2.05 notes describe. What the 9 means, and what a receiving client does with a one-byte payload, is **unresolved**. An emote-form payload with a real image file in the receiver's folder was not tried. (Later read: 2.05 does not use a four-field form; it sends the same tagged image list, see `../id24-205.md`.)

C. PLO_SERVERWARP (178)

Table: `version-profiles/lab-serverwarp178.tsv`. Real 2.31 client only.

The chain [request synthetic, rest real] - A client `PLI_SERVERWARP` (41) carrying the target server's name was injected on the 2.31 connection. The real game server passed the name to the real lab listserver, which answered. The game server wrapped the answer as 178 for the client. The wire text had four comma fields: `<name>,<name>,<host>,<port>`. - That answer comes from the **open-source listserver reimplementation**. What a real service sends is **unobserved**, although the 2.31 client accepted this one.

What the client does on 178 [observed] - It closes the connection to the current game server (FIN). A few milliseconds later (2 to 3 ms) it opens a TCP connection to the host in field 2 and the port in field 3, and logs in again by itself: login frame type 5, a new random key, its normal version token, the same account and the same stored password. - There is **no new listserver connection** and no server-list screen. The second game server accepted the login and sent its usual login sequence.

Field order [synthetic 178 frames] | Fields sent (field 0 first) | Client reaction | |---|---| | decoy host, name, host, port | connected to the host in field 2 and the port in field 3; no attempt to the decoy. **Field 0 is unused.** | | any, host, port, name | no TCP connection; old connection closed; "The connection couldn't be established!" with Reconnect and Open serverlist | | any, name, port, host (port and host swapped) | same failure | | any, a bogus name, host, port | connected normally. **Field 1 is a name and is not needed to connect.** | | any, name, host (three fields) | ignored: connection stays, nothing happens. Consistent with "at least 4 fields". |

- So on the wire: **field 0 unused, field 1 = server name, field 2 = host, field 3 = port.** This matches the decompile reading in `02-server-to-client-layouts.md`.
- **Unresolved:** whether field 1 is used for anything else (a title text, say). Not tried: 2.22, a host name instead of an IP in field 2, a port above 65535, extra trailing fields. A script-initiated `serverwarp` did not run in the lab (see A), which is why the request was injected.

What to implement (Seihai, 2.31 and 2.22 profiles)

1. **2.31:** send `46 = GCHAR 1` once per level entry, immediately before the first props frame of the new level. In the same frame as the warp request when the level is cached; in the first client frame after the server's level answer (6 or 14) otherwise; at login, after the `PLI 35` messages and before `PLI 44`. Drop the "login one-shot" idea.

2. **2.22:** do not send 46. If the tamper check is enabled, send 39 in the same frame right after each client-initiated `PLI_LEVELWARP` or `PLI_LEVELWARPMOD`, not after a server-side warp.
3. **178:** parse four comma fields, use fields 2 and 3, close the connection, reconnect to that host and port with a fresh login (new key, same credentials). Ignore the message with fewer than four fields. No listserver step.
4. **24 and 32:** treat 32 as `GSHORT id` + opaque bytes. Do not crash on a one-byte or empty payload. 2.31 sends 24 = 9 always; 2.22 was seen sending it only while another player was in the level (condition not isolated).

`client-internals/decomp-findings-2026-10-24-c/README.md`

Decompile mining, batch c: Graal 2.00 - 2.1 test, NewWorld, NewMain (Ghidra + IDR), 2026-10-24 (Asia/Tokyo)

Derived facts only. No raw decompile text, no addresses, no generated function names, no code. Raw exports are private (outside every repo). Nothing was pushed, no public server was contacted, the lab was not touched.

Scope: 20 Drive builds decompiled with Ghidra 12.2 and named with IDR under Wine. Supersedes the 2.00/2.05 parts of batch b where they conflict (see 10).

Files

file	content
01-toolchain-and-coverage.md	what was decompiled, Delphi/dispatcher fix, IDR, method, limits
02-login-token-and-version.md	tokens per build, date reading, login shape, language packet
03-framing-and-encryption.md	framing, zlib.dll in the 2.02 family, junk-byte status
04-plo-server-to-client.md	handled ids per build, adds/removes, 26 history, layouts
05-pli-client-to-server.md	sent ids per build, meaning of 35-38, id 6
06-handler-changes-by-build.md	behaviour changes 2.00 -> 2.1 and revision differences
07-udp-channel.md	UDP channel in 2.0x, ids per build
08-test-builds-newworld-newmain.md	2.1 test2/test3, NewWorld, NewMain
09-features-classes-vocabulary.md	vendor changelog vs binary evidence, classes, script strings
10-open-questions-and-corrections.md	corrections to earlier notes, open items, Seihai implications (items 1, 3, 4 closed by the version-profiles layout TSVs)
coverage_by_build.tsv	per build: size, function count, IDR, id counts
plo_handled_ids_by_build.tsv	server-to-client ids handled
pli_sent_ids_by_build.tsv	client-to-server ids with a send
id_changes_between_builds.tsv	adds/removes per step
version_tokens.tsv	login token and post-login send
plo_decode_signatures.tsv	heuristic field layouts per id (not authoritative for long handlers)
handler_shape_changes.tsv	per id, per step: similarity of normalized handler body
classes_by_build.tsv	class counts and class changes
script_vocab_deltas.tsv	script/identifier string changes

Top facts

1. Tokens: 2.00 GNW24011, 2.01 GNW23021, 2.02 GNW15041, 2.02 rev2 GNW20041, 2.03 GNW06051, 2.04 GNW24061, 2.05 and 2.1 test GNW03081, NewWorld G2N15041, NewMain GNW08011.

2. PLO sets: 53 (2.00) -> 55 (2.01: +46, +156) -> 56 (2.02: +47) -> 58 (2.03: +48, +165) -> 58 (2.04: +166, -26) -> 59 (2.05: +168) -> 61 (2.1 test: +49, +169).
3. PLI sets: 35 (2.00) -> 37 (2.01: +35, +36) -> 38 (2.02 rev2: +37 language) -> 39 (2.03: +38). No id removed. Id 6 is a real sender.
4. Revisions of one version are identical in game logic. 2.02 rev2 is the exception (new token + language packet).
5. 2.02, rev1 and rev2 import zlib.dll; all other builds carry zlib inside the exe.
6. UDP ids: 2.00 already sends 10-14, 16-20; 25 from 2.01, 26 from 2.03; PLI 36/38 are their TCP twins.
7. 2.1 test2/3: same PLI as 2.05, new PLO 49 PLAYERWARP2 and 169 UPGRADEURL. NewWorld beta1 follows the 2.05 line, beta2 the 2.1 line; NewMain is an older separate branch (2.00 minus freeze).
8. The 2.1 test2/test3 decompiles are lossy (the decompiler drops some arguments and calls). Treat every 2.1 test fact here as structure only (ids, field counts, which handler changed), not argument-level.

`client-internals/decomp-findings-2026-10-24-c/01-toolchain-and-coverage.md`

01 Toolchain and coverage

Derived facts only. No raw decompile text, addresses or generated function names are in this folder. Raw exports are kept private and are not in any repo. Run date: 2026-10-24 (Asia/Tokyo).

Result

- All 20 requested builds were decompiled with Ghidra 12.2 headless (JDK 25), at most 2 jobs at a time. No job hit the 7200 s limit and no job failed. Per-build table: `coverage_by_build.tsv`.
- Builds: 2.00 (main) and revs 1-3; 2.01 and rev1; 2.02, rev1, rev2; 2.03; 2.04; 2.05, rev1, rev2; 2.1 test2 and test3; NewWorld beta1/beta2; NewMain 24/26.
- 2.05 rev2 and 2.00 main already had decompiles. They were re-run only as a diff baseline (same pipeline, so differences are not tool artefacts).
- Each export holds about 5,480-5,720 functions. Most are VCL/RTL, JPEG library and DirectX wrappers. About 100 are game classes.

Why plain Ghidra output was not enough (Delphi finding)

- Every build is a Delphi 3/4/5-era PE (CODE/DATA/BSS sections, VCL RTTI). Plain auto-analysis missed about 520 functions per build, including the whole server-to-client packet switch and some real senders.
- Fix, in two Ghidra scripts run per build before export:
    1. Create functions at standard Delphi prologue sites that analysis skipped.
    2. Rebuild the packet dispatcher as one function from its jump table plus the byte index table in front of it. The table is indexed by `id` and has 0x9b..0xa9 entries depending on the build.
- Without step 2 the handled-id set per build cannot be read from the decompile at all. With it, the sets in `plo_handled_ids_by_build.tsv` match the 2.05 set already documented in the repo (59 ids), which is the check that the method is sound.
- Lesson for any later pass on another Delphi build: always check that the dispatcher exists as a function and that the jump-table size matches the "last index" in `plo_handled_ids_by_build.tsv`.

IDR (Interactive Delphi Reconstructor) status

- IDR 2.x runs under Wine 10 on a private virtual display, driven by xdotool. It works for every build. The first attempt on 2.1 test2 failed (a stray click opened a form preview) and the retry succeeded.

- 20 of 20 IDC name tables were produced (about 6,590-6,730 names each: RTTI classes, RTL, VCL and the application's own methods). They were substituted into the Ghidra C export. About 27-29k substitutions per build.
- Value: the dispatcher cases, the login function and the senders become identifiable. Method names of the game's own classes (TAdventureClient, TPlayer, TAdventureLevel, ...) are the only game-specific names IDR can recover. Packet handlers themselves are anonymous bodies in one switch, so their names come from gs2lib (`PLO_*` by id) and not from IDR.

Method for the tables

- PLO ids handled: dispatcher jump table minus the shared default arm.
- PLI ids sent: the union of (a) constant-id send calls found in the decompile and (b) an assembly scan for the same call pattern. The union is needed because one 2.03 sender (id 80) is missing from the decompile but present in the assembly. Id 6 (chat/TOALL) is a real sender in every build. The earlier "may be a false positive" remark in batch b was wrong.
- Handler layouts: heuristic scan of how a handler reads the payload (GCHAR/GINT2/GINT3 reads, rest-of-string copies, length guards). It is exact for simple handlers and incomplete for long ones. `plo_decode_signatures.tsv` records the result and is not authoritative for long handlers.
- Handler change detection: normalized statement shapes (names, big numbers and casts removed), compared with a sequence-similarity ratio. See `handler_shape_changes.tsv`. Small bodies (4-12 statements) change shape from compiler noise, so only large drops in similarity count.

Limits

- The 2.1 test decompiles drop some arguments and calls (the Ghidra decompiler loses some register arguments in that compiler version). Structural facts (ids, field counts, which handler changes) are reliable. Exact argument values are not.
- The raw exports are private. Re-running needs the lab's private tool folder (Ghidra, IDR; not in a repo).

`client-internals/decomp-findings-2026-10-24-c/02-login-token-and-version.md`

02 Login token and version string

All tokens come from the binaries (the 8 characters are built one byte at a time from immediates, so a text search misses them). Table: `version_tokens.tsv`.

Token per build

build	token
2.00, rev1, rev2, rev3	GNW24011
2.01, 2.01 rev1	GNW23021
2.02, 2.02 rev1	GNW15041
2.02 rev2	GNW20041
2.03	GNW06051
2.04	GNW24061
2.05, rev1, rev2	GNW03081
2.1 test2, test3	GNW03081 (same as 2.05)
NewWorld beta1, beta2	G2N15041 (different prefix, same digits as 2.02)
NewMain 24, 26	GNW08011

Reading the digits [inferred]

- The shape is prefix + dd + mm + y (day, month, last digit of the year). Read that way, 2.00 is 24 Jan 2001, 2.01 is 23 Feb, 2.02 is 15 Apr, 2.02 rev2 is 20 Apr, 2.03 is 6 May, 2.04 is 24 Jun, 2.05 is 3 Aug. All sit inside the year-2001 changelog (`newfeatures2001.txt`) and rise in order, so this is very likely a build date.
- NewMain (08011 = 8 Jan) is older than 2.00 by this reading, which fits a branch that was frozen before the 2.00 release.
- NewWorld beta carries "G2N" and the 2.02 date, so it branched from the 2.02 code line. Its class list agrees (see 09).
- Revisions within one version do not change the token (2.00 rev1-3, 2.01 rev1, 2.02 rev1, 2.05 rev1/rev2) except 2.02 rev2, the only rev that changes both token and login behaviour. A server allow-list therefore needs one entry per version, plus an extra entry for 2.02 rev2.

Login string shape (unchanged across all 20 builds)

- Type 0 packet: one random key byte (GCHAR of key & 0x7F), the 8-character token, GCHAR(len) + account, GCHAR(len) + password. Account and password are plain.
- Straight after it, 2.02 rev2 and every later 2.x build (also NewWorld) send PLI 37 with the language string. 2.00 to 2.02 rev1 do not. This matches the vendor changelog for 2.02 rev1 ("the server knows the language of the client", planned) and the actual send shows up in rev2.

What changed in the login function between revisions

- 2.02 rev1 to rev2: only the token and the extra id-37 send. No other login changes.
- 2.00 rev3: a change in a player-creation routine and the tab-control handler, not in login.
- 2.05 rev1/rev2: changes only in main-form creation and mouse handling.

Server consequence

A lab or Seihai server that wants all of these clients must list all token strings above. Tokens never contain a version number as such, only the date code, so the server cannot derive a build's capabilities from the token alone: see `04` and `05` for what each build can send/handle.

`client-internals/decomp-findings-2026-10-24-c/03-framing-and-encryption.md`

03 Framing and encryption (2.00 - 2.1, NewWorld, NewMain)

What is the same everywhere

- Message = GCHAR(id) + payload + newline; id = byte - 0x20. GCHAR = byte - 0x20. GINT2 = (b0-0x20)*0x80 + (b1-0x20). GINT3 adds a third byte with multipliers 0x4000, 0x80, 1. These read patterns are used by handlers in all 20 builds with the same constants.
- Frames on the wire are a 2-byte big-endian length followed by a zlib body; the client's outgoing messages are collected and flushed as one zlib block. The shared Borland LCG constant occurs in every build (it is also used by the Delphi random-number routine, so by itself it does not prove the junk-byte step; the junk-byte description in batch b for 2.00 still applies). No build changes the flow in a way that needs a new decoder.

- PLO 100 (RAWDATA, GINT3 length) is handled in every build from 2.00 on, including NewWorld and NewMain. Ids 101/102 never appear as handled ids in any build (they travel behind 100).

Where zlib comes from (the one real difference)

builds	zlib source
2.00 rev1-3, 2.01, 2.01 rev1	compiled-in Pascal zlib unit (z_stream record visible in names)
2.02, 2.02 rev1, 2.02 rev2	external `zlib.dll` (import, with deflate/uncompress/deflateInit thunks)
2.03 and later (2.04, 2.05, 2.1 test, NewWorld, NewMain)	compiled-in Pascal unit again

- The 2.02 family therefore needs zlib.dll beside the exe, or a Wine override, to run. All other builds run without it.
- The change is link-level only; the wire format is the same.
- Possible reason [inferred]: a short-lived switch to the DLL during 2.02, then back. No changelog line mentions it.

Encryption generations

- All 20 builds send type 0 login with the 8-character token in the first 8 bytes. A server that treats "first 8 bytes is a known token" as generation 2 (zlib, no cipher) serves them. The 2.05 behaviour (insert one junk byte per message after the first) is described in batch b for 2.00 and 2.05 and was not seen to change in 2.01-2.04.
- No build before 2.1 has bz2 or a per-frame compression-type byte. The 2.1 test builds also do not (they behave like 2.05 here).
- NewWorld beta1/2 and NewMain 24/26 use the same transport and the same login shape as 2.00-2.05.

Not verified here

- Whether the gen-3 junk-byte step is byte-for-byte identical in 2.01-2.04 (the call structure matches 2.00 and 2.05, but I did not diff the exact insertion index formula per build).

`client-internals/decomp-findings-2026-10-24-c/04-plo-server-to-client.md`

04 Server-to-client ids (PLO) per build

Tables: `plo_handled_ids_by_build.tsv` (sets), `id_changes_between_builds.tsv` (adds/removes), `plo_decode_signatures.tsv` (field layouts, heuristic). Names are gs2lib's.

Set growth

step	added	removed
2.00	53 ids: 0-45, 75, 100, 150, 151, 153, 154, 155	
2.00 rev1-3	none	none
2.01	46 HITOBJECTS, 156 SETACTIVELEVEL	
2.01 rev1	none	
2.02	47 STAFFGUILDS	
2.02 rev1, rev2	none	
2.03	48 TRIGGERACTION, 165 MOVE	
2.04	166 SHOWIMGNPC	26 NPCACTION

step	added	removed
2.05, rev1, rev2	168 (UNKNOWN168)	
2.1 test2	49 PLAYERWARP2, 169 UPGRADEURL	
2.1 test3	none	

- Dispatcher bound (last index): 0x9b (2.00-2.00r3), 0x9c (2.01-2.02), 0xa5 (2.03), 0xa6 (2.04), 0xa8 (2.05), 0xa9 (2.1 test). It always grows with the highest handled id.
- Correction to batch b note 04: 2.04 drops 26 only. 25 (SIGNATURE) stays in every build. The earlier wording "swaps 26 for 25" was wrong.
- Id 26 (NPCACTION, absent from the repo's 2.05 table) exists in 2.00-2.03. It reads GINT3 npc id and the rest, and only acts when the player's level is the active one. It has no handler in 2.04 and later, which agrees with the repo's "no jump-table entry for 26 in 2.05".
- Id 168 has an almost empty body (client-side reset only). In the 2.00-2.05 family it is first handled in 2.05 (all three 2.05 builds), consistent with batch b's 2.05 note that the login server sends it.
- 156 SETACTIVELEVEL is handled from 2.01 on. It stores the active-level reference by looking up the raw level name. The handler was reworked (much shorter) in 2.03.

Active-level gating

- Only id 26 is gated on "player's level == active level" in the 2.0x builds, by my scan. For later ids (46-48, 165, 166) the batch-b 2.17+ analysis already lists the gating; I did not re-derive it per 2.0x build.
- Because 156 only appeared in 2.01, a 2.00 client has no active-level concept. Level-addressed packets in 2.00 act on the current level only.

Layouts that are stable and verified by structure

(GINT/GCHAR counts; field meanings come from gs2lib. "len>N" means a minimum payload length guard.) - 3 NPCPROPS, 24 NPCMOVED, 29 NPCDEL, 150 NPCDEL2, 165 MOVE, 166 SHOWIMGNPC: GINT3 npc id first. - 8 OTHERPLPROPS, 11 BOMBADD, 19 ARROWADD, 20 FIRESPY, 21 THROWCARRIED, 32 SHOWIMGPLAYER, 36 EXPLOSION, 37 PRIVATEMESSAGE, 40 HURTPLAYER, 46 HITOBJECTS, 48 TRIGGERACTION: GINT2 player id first, then the rest. - 42 NEWWORLDTIME: GCHAR + GINT3 (length guard >3); present since 2.00 and in every build, including NewMain. - 13 TOALL: GINT2 sender + GCHAR + string. The 2.00 and 2.01 decodes looked different only because the scan was incomplete for the long handler.

Handler behaviour changes

See `06-handler-changes-by-build.md` and `handler_shape_changes.tsv`.

Missing in NewWorld and NewMain

See `08`.

`client-internals/decomp-findings-2026-10-24-c/05-pli-client-to-server.md`

05 Client-to-server ids (PLI) per build

`pli_sent_ids_by_build.tsv`; adds/removes in `id_changes_between_builds.tsv`. Names are from the repo's `packet-ids.md`.

build	ids with a direct send	count
2.00 (and rev1-3)	0-24, 26-30, 32-34, 80-81	35
2.01, 2.01 rev1, 2.02, 2.02 rev1	+ 35, 36	37

build	ids with a direct send	count
2.02 rev2	+ 37	38
2.03 to 2.1 test3	+ 38	39

- 35 = ADJACENTLEVEL (matches the 2.01 changelog "levels are preloaded when you walk on the map"). The payload is a string taken from the neighbouring level object, completed with a second computed string [layout not decoded].
- 36 = HITOBJECTS (2.01; script variable `hitobjects` appears in the same build). [inferred] It is the TCP twin of UDP id 25: the same function first sends id 25 through the peer-channel wrapper, then repeats the message over TCP under 36 when a flag says the server needs it. That agrees with the routing rule in `udp-p2p.md`.
- 37 = LANGUAGE (2.02 rev2; also sent once right after login).
- 38 = TRIGGERACTION (2.03; twin of UDP id 26 in the same way as 36/25 [inferred]). Matches the 2.03 changelog.
- 6 = TOALL (chat) is a real sender in every build. Chat text is clipped to 200 characters before sending. The batch-b remark that 6 could be a scan artefact is withdrawn.
- 80/81 (profile get/set) are present from 2.00 on.
- No direct constant send exists for 25, 31, or any id from 39 up in any 2.0x or 2.1-test build. So the tamper check (39) and the later periodic ids are not part of this family. 42 and 45 never appear.
- No PLI id was removed anywhere in the 2.00 to 2.1 line.

NewWorld / NewMain

- NewWorld beta1/2: 34 ids. They have 35-38 but lack 7 (HORSEADD), 11 (THROWCARRIED), 15 (BADDYPROPS), 16 (BADDYHURT) and 17 (BADDYADD): horses and baddies are removed from the game in NewWorld.
- NewMain 24/26: 35 ids, same as 2.00 (0-24, 26-30, 32-34, 80-81). They lack 35-38, in line with their older token.

`client-internals/decomp-findings-2026-10-24-c/06-handler-changes-by-build.md`

06 Handler behaviour changes, 2.00 -> 2.1

Basis: normalized-shape comparison of every handled id between consecutive builds (`handler_shape_changes.tsv`, similarity below 0.93 only), checked against the vendor changelog. A shape change means different statements, not necessarily different wire behaviour. Small handlers (4-12 statements) shift shape from compiler noise; only large drops are listed below.

Revisions of the same version

- 2.00 rev1/rev2/rev3, 2.01 rev1, 2.02 rev1/rev2, 2.05 rev1/rev2: identical handler shapes for every id except one: 2.05 rev1 has one extra constant assignment in OTHERPLPROPS (id 8) that 2.05 and 2.05 rev2 do not have.
- The revisions differ in the VCL/RTL layer, not in game logic. 2.00 rev2, 2.02 rev2 and 2.05 rev1 each carry about 60 changed VCL/RTL functions (the multi-monitor wrapper set appears in rev2 builds; rev3 rolls part of it back). The only game-class changes are:
    - 2.00 rev1: one added options-form key handler.
    - 2.00 rev3: player creation and the tab-control handler.

    - 2.02 rev2: login (token + language) and the player-list history click.
    - 2.05 rev1/rev2: main-form creation and mouse-move handling; rev2 also a level-creation routine and server-list form creation.
- So for protocol work any rev of a version can stand in for the others, except 2.02 rev2 (different token, extra id 37).

2.00 -> 2.01

Biggest handler rewrites in the family (gs2lib names): - LEVELBOARD (0) and PLAYERWARP (14), WARPFAILED (15): the client no longer does the "wait for level, then request level" steps inside the handler. This is the code side of "levels are preloaded when you walk on the map" in the changelog. - LEVELNAME (6): shortened. - BADDYHURT (27): a distance/direction-based calculation before the effect is removed. - FREEZEPLAYER2 (154) / UNFREEZEPLAYER (155): go through a shared helper with a flag, instead of setting the field directly. 155 shrinks to one short step. - HASNPCSERVER (44): stores a different constant than in 2.00 (value 11 instead of 1). - New handlers: HITOBJECTS (46), SETACTIVELEVEL (156).

2.01 -> 2.02

- Only SAY2 (153) and PLAYERWARP (14) change shape. 47 STAFFGUILDS is new (list stored as comma text).
- The big 2.02 features (hats via gani attributes, signs with images, string parsing, free-server restrictions) are client-side script/UI and do not add any packet id other than 47.

2.02 -> 2.03

- NPCMOVED (24) grows (31 to 48 statements) and TOALL (13) changes slightly. That is the NPC `move` command and the pushed/pulled actions.
- SETACTIVELEVEL (156) shrinks to a single step.
- New: TRIGGERACTION (48), MOVE (165).

2.03 -> 2.04

- NPCMOVED grows again (48 to 55). MOVE (165) shrinks (17 to 9). SHOWIMGNPC (166) is new: matches "showimg works server-side, layer 4 works too".
- 26 NPCACTION is removed.
- Many other ids show only compiler-level churn.

2.04 -> 2.05 (all revs)

- SHOWIMGNPC (166) loses about 8 statements. UDP receive wrapper loses a bit-test step (purpose not resolved).
- New: id 168 (empty-ish body).
- Everything else is churn-level.

2.05 rev2 -> 2.1 test2

- Large churn in almost every handler. Treat as weak evidence: the 2.1 compiler output is lossy, and the new imaging engine (see `09`) touches many calls. Reliable facts are the id sets only (new 49, 169).

- PLAYERWARP2 (49): x2/y2 half-tile coordinates, an ignored z, two gmap cell bytes and the level name, same structure as the 2.17-2.31 layout in batch b. Tile positions are halved on receive.
- UPGRADEURL (169): raw text stored for the reconnect/upgrade button.

2.1 test2 -> test3

- LEVELBOARD (0), LEVELNAME (6) and PLAYERWARP (14) gain "is the incoming level the one already loaded" checks, so the client avoids reloading the same level. Same direction as in 2.31 (batch b noted that LEVELNAME compares with the current level).
- SAY2 (153) gains one extra call. The change to 31 is a constant-type difference only.
- No id added or removed.

`client-internals/decomp-findings-2026-10-24-c/07-udp-channel.md`

07 UDP peer channel in 2.0x

Compare with `Graal_analysis_draft/protocol/udp-p2p.md` (2.05 rev2).

- Every one of the 20 builds imports the same socket calls (socket, bind, sendto, recvfrom, ioctlsocket, getsockname, inet_addr, gethostbyname). The UDP channel therefore exists from 2.00 on, in NewWorld and NewMain too.

- Per build there is one send wrapper and one receive wrapper. The send wrapper has the same shape (about 40 normalized statements) in every build. The receive wrapper has the same shape (about 82 statements) from 2.00 to 2.04 and NewMain, shrinks to 74 in 2.05 and NewWorld beta1, and to 72 in 2.1 test2/test3 and NewWorld beta2.

- The 2.05 change is a removed bit-test on a length value (alignment or sanity step, purpose unresolved) and a field offset; nothing about the datagram layout changes. Datagram shape in 2.00-2.04 is still seq byte + XORed payload [structure confirmed on 2.00-2.04 by the same call structure, not by per-id decode]. #### UDP ids sent per build (constant-id call sites of the peer-send wrapper) | build | ids found | |---|---| | 2.00 (and revs) | 10, 11, 12, 13, 14, 16, 17, 18, 19, 20 | | 2.01 | + 25 (hit objects) | | 2.03 | + 26 (trigger action) | | 2.04 | 11, 16, 17, 19, 20, 25, 26 found (other call sites go through a changed wrapper form; not recovered) | | 2.05 | the 2.05 rev2 set from `udp-p2p.md`; the scan found only the movement sender (id 0) |

- These were partial in the first scan: ids 0 (movement), 15 (showimg) and 22 (chat) use a broadcast helper (prepend GCHAR id, send to every peer with a UDP port) rather than a naked constant at sendto. Manual read recovered them for 2.00 and 2.05; see `version-profiles/udp-ids-200-204.tsv`. Receive switch cases are wire bytes 0x20 / 0x2f / 0x36.

- The additions match the TCP ids: UDP 25/26 appear in the same builds as PLI 36 (2.01) and PLI 38 (2.03), which supports the "UDP + TCP twin" reading in `05`.

- So the 2.00 client has movement, showimg, chat, bomb, arrow, firespy, explosion, horse, throw and item UDP messages (ids 0, 10-20, 22); hit objects arrive in 2.01 and trigger action in 2.03.

- A 2.00 client has no hit-objects message on either channel.

client-internals/decomp-findings-2026-10-24-c/08-test-builds-newworld-newmain.md

08 The 2.1 test builds, NewWorld betas, NewMain

2.1 test2 and test3

- Same login token as 2.05 (GNW03081), same login shape, same PLI set as 2.05 (39 ids, 0-24, 26-30, 32-38, 80-81). No new client-to-server id.
- Two new server-to-client ids: 49 PLAYERWARP2 and 169 UPGRADEURL (61 ids handled, bound 0xa9). Same two ids appear in NewWorld beta2 and are in the 2.17-2.31 layouts documented in batch b, so these builds are where they started.
- test2 to test3: same id sets. Differences are the "skip reloading the already-loaded level" checks in LEVELBOARD, LEVELNAME and PLAYERWARP, plus one extra call in SAY2. Vendor changelog adds snapshot-level lighting fixes and smilies in PMs (`levels/smilies/smiliesdef.txt`, lines `code,filename`). Those are client-only.
- The exe is about 140 KB smaller than 2.05 rev2 despite more functions, because of the rewritten image engine (the changelog says the image handling no longer uses Windows GDI).
- Editor classes changed: the person/NPC/show-code windows were merged into a tabbed editor (class list in `classes_by_build.tsv`).
- Script additions (strings): `ascii`, `defnpc` (predefined npcs from `defnpc*.txt`), `playeronline`, `startinfullscreen`, `tiles` (test2); `groundheights`, `waterheight`, `playerhp`, `playermaxhp`, `smiliesdef.txt` (test3). Also see `script_vocab_deltas.tsv`.

NewWorld beta1 / beta2

- Token G2N15041. Branch point: the 2.02 code line (token digits, class list with the old person-editor windows, same PLO bound as 2.05 for beta1).
- Beta1 server-to-client set: 53 ids. Compared with 2.05 rev2 it lacks 2 (BADDYPROPS), 4 (LEVELCHEST), 17/18 (HORSEADD/DEL), 154/155 (freeze/unfreeze), 49 and 169, and (like 2.04+) 26; it still has 156, 165, 166, 168.
- Beta2 adds 49 and 169 (55 ids) and switches to the 2.1 editor classes. In other words beta2 follows the 2.1 test branch; beta1 follows 2.05.
- Client-to-server: 34 ids. Same as 2.1 test but without 7 (HORSEADD), 11 (THROWCARRIED), 15 (BADDYPROPS), 16 (BADDYHURT), 17 (BADDYADD).
- Script vocabulary: new `nw*` time variables (nwyear, nwmonth, nwweek, nwweekday, nwday, nwhour, nwmin, nwtime), `layer`, `arraysnw.dat`, `bodycolors.txt`, `bodynw.png`, `controlnpc.txt`, `selectedsword`, `playerhearts`, `playerfullhearts`, plus string-list helpers (addstring, insertstring, deletestring, removestring, sarraylen, lindexof) in beta2. Dropped: the Graal classic baddy/animal names (blue soldier, gray soldier, dragon, frog, ...) and `arrays.dat`.
- Confirms that the NewWorld "world" is a different content stack (new body images, own array file) on top of the unchanged 2.0x protocol, with the NEWWORLDTIME packet (42) already present in 2.00.

NewMain 24 / 26

- Token GNW08011. PLO set: 51 ids, 0-45, 75, 100, 150, 151, 153 only. That is the 2.00 set without 154, 155. PLI set: 35 ids, the 2.00 set.
- NewMain 24 and 26 have identical handler shapes and an identical login. They differ only in resources and a few VCL calls.

- NewMain pulls in the Delphi DDE classes (TDdeMgr etc.) and keeps the person/NPC/chest/baddy editor windows, consistent with an older, pre-2.00 code line.
- Handler shapes differ from 2.00 more than any 2.0x revision does (many ids below 0.7 similarity), so NewMain is a separate branch, not a rev of 2.00.

`client-internals/decomp-findings-2026-10-24-c/09-features-classes-vocabulary.md`

09 Features, classes, vocabulary

Tables: `classes_by_build.tsv`, `script_vocab_deltas.tsv`. Vendor source: `newfeatures2001.txt` in each version folder (2.01-2.05 and the 2.1 tests). The 2.00 builds and NewWorld/NewMain have no changelog.

What each step adds (vendor text + binary evidence)

step	feature	binary evidence
2.01	levels preloaded when walking on the map; map-reading animation; showimg artefact fixes; chat text from `setplayerprop #c` stays 3 s	PLI 35 and PLO 156 appear; warp/board handlers simplified; script strings bomy_walk/bomy_idle, maps1-3, hitobjects, playerreading, playerdies/hurt
2.01	`hitobjects`	PLI 36, PLO 46, UDP 25
2.02	hats via gani attributes (5 attributes read from the player account), `#i()` images in signs, string parsing (strlen, startswith, indexof, `#e`, `#T`, tokenize/tokenize2/tokenscount/`#t`), `local.` flags are not sent to the server, flags issparringzone/nopkzone, free-server restrictions, HTML window navigation buttons, staff-guild list editable by server owners	PLO 47; new strings for heads/bodies/shields/hats directories, `local.`, webgifs, weblevels, profiles; zlib.dll import (see 03)
2.02 rev1	speed-cheat check fix, history without web bar, better hacking protection, server learns the client's language	server language is sent starting rev2 (PLI 37)
2.03	`triggeraction`, `actionpushed/actionpulled`, `move dx,dy,time,options`	PLO 48 and 165, PLI 38, UDP 26, NPCMOVED handler grows; strings pushed/pulled/vecx/vecy/triggeraction/paramscount/movementfinished/onmapx/onmapy
2.04	`replacegani`, `addtiledef`, `addtiledef2`, `removetiledefs`, `setshape`, `setshape2`, `attachplayertoobj`/`detachplayer` with playerattachid/type and levelorgx/levelorgy, showimg text and server-side showimg, weaponscount/`#w`/`#W`, enable/disableselectweapons, bomy_kick	PLO 166, 26 removed; strings tiledefs, tileobjects, shapetiletypes, playerattached

step	feature	binary evidence
2.05	maps via `setmap` applied at once, window title with server name, script-editor format button, `.nw` text level format (GLEVNW01), array tiletypes for setshape2	PLO 168; strings updateterrain, loadmap, getz/playerz, setbacktile2
2.1 test	new tile art, new start/server-list window (F8), new image engine, `ascii()`, `tiles[x,y]`, predefined-npc editor with `#COMMENTS`/`#EDIT`, smilies in PMs, tabbed editor	PLO 49 and 169; TPageControl/TTabSheet/TMemo added, person/show-code windows removed

Classes

- 186 classes in 2.00 through 2.03. +1 in 2.04 (an object-set class; matches "support for object sets"). +1 in 2.05 (class TADP; purpose not identified). 2.1 test swaps 4 editor classes for 4 tabbed ones (188 total).
- NewWorld beta1 = 186 classes, the same set as the 2.0x line (without the 2.04 and 2.05 additions); beta2 changes the editor windows like 2.1 test. NewMain has 193 (adds DDE classes, chest/baddy editors).

Vocabulary (script/identifier strings, resources excluded)

See `script_vocab_deltas.tsv` for exact lists. Takeaways: - Everything the changelog calls a new script command or variable appears in the binary's string table in the same build, with no extra undocumented identifier in 2.01-2.05 other than `actionplayer`, `lighteffectsenabled` (2.03) and `bomy_*` animation names. - 2.01 drops `openurl` from the string list and 2.02 drops `error`/`z_stream8` (then 2.03 brings them back and drops the zlib.dll strings). These are string-table moves, not feature removals: the 2.02 changelog still describes `openurl`/`openurl2`.

`client-internals/decomp-findings-2026-10-24-c/10-open-questions-and-corrections.md`

10 Open questions and corrections

Corrections to batch b / earlier notes

- Batch b note 04: "2.04 swaps 26 for 25" is wrong. 2.04 only drops 26; 25 is present in all builds.
- Batch b note 04: "id 6 may be a false positive of the scan". Wrong: 6 (TOALL/chat, clipped at 200 characters) is a real sender in all 20 builds.
- Batch b treated 2.00 as one build; the 4 builds (2.00, rev1, rev2, rev3) have identical handler shapes and one token. The only differences are VCL/RTL layer changes and small UI-class edits.
- Repo `packet-ids.md` says PLO 26 NPCACTION has no handler in 2.05: confirmed, and the earlier history is now known: handled and active-level gated in 2.00-2.03, removed in 2.04.

Open

1. ~~Exact field layouts of the long handlers (PLAYERPROPS 9, OTHERPLPROPS 8, NPCPROPS 3, LEVELLINK 1) per build: the layout scan is too crude. Needs per-handler manual reading or a type-aware recompile of the decompile.~~ Done: `version-profiles/plo-playerprops-layouts.tsv`, `plo-otherplprops-layouts.tsv`, `plo-npcprops-layouts.tsv`, `plo-levellink-layouts.tsv` (builds

2.00, 2.04, 2.05 rev 2, 2.17.1, 2.22, 2.31). 2.04 rows are bracketed from the 2.00/2.05 endpoints plus batch-c shape/signature tables (2.04 `.c` not in the shared Drive Graal Decompiles folder).
2. 2.04 to 2.05 UDP receive wrapper: removal of one bit-test step. Purpose not resolved.
3. ~~Per-build UDP id sets for ids 0, 15 and 22 (movement, showimg, chat) for 2.00-2.04: the call forms were not recovered. Receiver-side sets for 2.00-2.04 are also not extracted (a second jump table the dispatcher rebuild did not cover).~~ Done: `version-profiles/udp-ids-200-204.tsv`. Ids 0/15/22 go through a broadcast helper (GCHAR id prepend + send to every peer with a UDP port), so the old constant-id scan missed them. Receive switch cases are wire bytes 0x20 / 0x2f / 0x36. Mid builds without a Drive `.c` are inferred from the 2.00 and 2.05 endpoints plus batch-c wrapper-shape notes.
4. ~~Whether gen-3 junk-byte insertion is byte-for-byte unchanged in 2.01-2.04. The call structure matches; the index formula was not diffed.~~ Done: `version-profiles/gen3-insertion-200-205.tsv`. 2.00 matches the known 2.05 formula (`it * 0x08088405 + k`, insert `((it>>16)&0x7F)+0x20` at `(it&0xFFFF) mod len`). With identical endpoints and batch-c's matching call structure, 2.01-2.04 are unchanged (those `.c` files are not on Drive).
5. The 2.1 test decompiles are lossy; argument-level facts about 49 and the changes to 0, 6, 14 there are structural only.
6. What PLI 36/38's TCP "second copy" is conditioned on (flag identity). Reading is [inferred] from the shared wrapper.
7. Purpose of TADP (new in 2.05) and of PLO 168 (empty client-side reset).
8. Whether the token digits really are a build date. The dates fit the changelog order for 2.01-2.05 but nothing in the binary names the format.
9. Not touched: tamper-check (PLI 39) is absent from this whole family; nothing here helps derive it.

For Seihai

- To support 2.00-2.1: allow-list the tokens in `02`; handle PLO ids per build set in `04`; PLI sets in `05`. A server that serves 2.1 test must be ready for 49 and 169 as server-to-client only; no new client-to-server id exists in 2.1 test.
- Do not send ids 46+ to a 2.00 client (it has no handler) and avoid 47+ for a 2.01 client, 48/165 for 2.02, 166 for 2.03, 168 for 2.04. Unhandled ids go to a silent default in every build, so the practical effect is lost information, not a crash.
- LEVELLINK, NPCPROPS, OTHERPLPROPS and PLAYERPROPS wire envelopes do not change across 2.00-2.31; only the accepted prop-id sets inside the streams grow. See the four `plo-*-layouts.tsv` tables.

`client-internals/decomp-findings-2026-10-24-d/README.md`

Decompile mining, batch d: sword damage and the respawn entry save, 2026-10-25 (Asia/Tokyo)

Derived facts only. No raw decompile text, no addresses, no generated function names, no code, no paths. The raw exports stay private. Nothing was pushed to Graal_analysis, no public server was contacted, the lab was not touched. Nothing here was seen on a wire.

Scope: the two rules that Seihai's combat and health/death work had left as placeholders. Read in the 2.05 rev 2, 2.17.1, 2.22 and 2.31 clients, with the 2.1 test builds and all other builds checked only for one constant (see 02).

Files

file	content
01-sword-damage.md	how the victim's client turns a hit into hearts, the sword-power byte, heal rule, hurt mode
02-respawn-entry-save.md	which level entry is saved as the respawn point, the gmap-list test, the name group (2.00 to 2.05)
03-hit-gates-and-open-points.md	extra gates on a hit (no-kill level, guild tag, attacker status, per-second limit), and what is still open
damage-rules.tsv	the damage rules, one row each, with the builds they were read in
hit-gates.tsv	every gate a hit must pass, in the order the client checks, per path
entry-save-rule.tsv	the save-entry decision table
name-group-constant-by-build.tsv	which builds contain the name-group delimiter constant
open-questions.tsv	what was not resolved

Top facts

1. A sword hit takes the attacker's sword power in half hearts, limited to -20..20. Shield, item levels and the attacker's AP are not read. Same in 2.05 rev 2, 2.17.1, 2.22, 2.31.
2. A negative power heals only a victim with AP 40 or more; below that it changes no hearts. Hurt mode (mode 3, 40 ticks) starts for every accepted hit, including power 0.
3. The sword-power byte on the wire is 0..4 for the default swords 1..4, and power + 30 (10..50) plus an image name otherwise.
4. A hit is dropped when the level ran the script command `noplayerkilling`, when the attacker's `(tag)` is in the victim's nickname, when the attacker's STATUS has bit 0x02 or 0x20, and when the same attacker was judged earlier in the same second of the day. The hurt packet (40) is stopped by the no-kill level too.
5. The respawn point is the last *saved* level entry. An entry is saved when there is no previous level, or when exactly one of {old level, new level} is in the current gmap's level list (the client multiplies the two list indices with -1 for "absent", so index 0 behaves like "absent": a quirk).
6. In 2.00 to 2.05, an entry that fails that test is still saved when the old and new levels are in different name groups. The name group is the file name without its folder, lower-cased, up to and including the last `§` or `$`; with neither, the group is empty. 2.17.1, 2.22 and 2.31 have no group test; the 2.1 test builds do not contain the delimiter constant.
7. With plain level names, the respawn point is therefore the login spot or the last gmap boundary crossing.

`client-internals/decomp-findings-2026-10-24-d/01-sword-damage.md`

Sword damage (victim's client)

Builds read: 2.05 rev 2, 2.17.1, 2.22, 2.31. The rule is the same in all four.

Who computes it

The victim's client. The attacker's swing reaches others only as props (position, direction, sprite); the hit's size comes from the attacker's SWORDPOWER prop as the victim last received it.

The sword-power byte (prop 8)

- Sender: the power is limited to -20..20. Power 0 sends byte 0 (no image). A power of 1..4 with the matching default image (sword1..sword4) sends the power itself, 1..4. Anything else sends power + 30 (10..50) followed by the image name.
- Receiver: bytes below 10 are the power; from 10 the power is the byte minus 30.
- The victim stores the power for each other player (a signed value) and uses it when a swing is judged.

The formula

Half hearts taken = the attacker's sword power, limited to -20..20. In hearts: power / 2. So a power-3 sword takes 1.5 hearts.

Not read: the victim's shield power, item levels, the attacker's AP, the hurt-power field of the hurt packet (that packet carries its own power; see below).

What happens to the hearts

The shared hurt routine takes a power in hearts and a knockback vector. It asks the heart-change routine for minus the power, then puts the player into hurt mode (mode 3, 40 ticks of 50 ms). - The heart-change routine ignores a positive change (heal) when the player's AP is below 40. So a negative sword power heals only at AP 40 or more, and hearts stop at the maximum. - It is also where the swim, editor/pause and death checks live: hearts reaching 0 start the death. - Hurt mode starts regardless of the sign or size of the power, including 0 and a heal that was ignored.

The hurt packet (id 40) and its power

`PLO_HURTPLAYER` carries the attacker, a knockback and a power byte (half hearts). The client limits the power to one heart (2 half hearts) before calling the same hurt routine; the attacker must be within 10 tiles. The 2.05 to 2.31 handlers differ only in small decode churn (the earlier batch c tables). The client's own id 26 send is the mirror image; its power is the attacker's choice, limited the same way.

Open

See 03.

`client-internals/decomp-findings-2026-10-24-d/02-respawn-entry-save.md`

Respawn destination: the level-entry save

The respawn moves the player to the *saved entry point* (level, x, y, direction). This note is about when the client saves it.

The decision, run after every level entry

Inputs: the name of the level just left (empty at login), the new level's name, and the current gmap's level list (a string list; -1 when a name is not in it or there is no gmap). 1. Level left is empty: save. 2. Compute index(new) * index(old). If it is negative: save. So exactly one of the two is in the list and at an index of 1 or more; an index of 0 gives a product of 0, so the first list entry behaves like "not in the list" (the client's quirk). 3. 2.00 to 2.05:

otherwise, compare the name groups. Different groups: save. Same group: do not save. 4. 2.17.1, 2.22, 2.31: otherwise do not save (no group test).

Saving stores the new level's name and the player's current x and y, direction as it is.

The name group (2.00 to 2.05)

Take the level name, drop everything up to the last path delimiter, lower-case it, then take the text up to and including the last § (byte 0xA7) or $. If there is neither, the group is empty, and two empty groups are equal. The extension stays in the name.

Effect: two plain names such as `a.nw` and `b.nw` are always the same group. The test only separates names that use the delimiters.

Which builds have the group test

Read in the code, step by step, for all eleven 2.00 to 2.04 builds: 2.00 and its revisions 1 to 3, 2.01 and rev1, 2.02, rev1 and rev2, 2.03, 2.04. Each has the same decision as 2.05 rev2, in the same order: the empty-name test, the product of the two list indices, then the name-group test, then the save. The name-group routine is the same instruction for instruction in all eleven builds and in 2.05 rev2. (Confirmed, from the machine code.)

One difference: 2.05 returns -1 for both indices when there is no current gmap. In 2.00 to 2.04 the list is taken straight from the client's state with no such test. Whether that list is simply empty when there is no gmap, which would give the same -1, was not traced. (Inferred, not tested.)

2.17.1, 2.22 and 2.31 have no group test. The 2.1 test builds 2 and 3 do not contain the delimiter-set constant, so they are expected to have none; their code was not read. A scan of the build files for that constant finds it in 1.37 to 1.41, 2.00 to 2.05, NewMain 24 and 26 and NewWorld beta 1. Builds before 2.00 were not read. See name-group-constant-by-build.tsv.

After the respawn

The respawn routine moves the player (same-level shortcut or a level change), then calls the same save routine once more, unconditionally. If the warp completed in the same call, this stores the respawn point itself. If an online level change finishes later, the stored point could be the spot where the player died. Not resolved.

Open

See 03.

`client-internals/decomp-findings-2026-10-24-d/03-hit-gates-and-open-points.md`

Extra gates on a hit, and open points

Gates

Beyond the swing sprite (S 6..10), the reach box, the victim's state (not dead, not in hurt mode, not swimming) and AP not 100, the client's shared hit-judging routine also requires: - the current level has not run the script command `noplayerkilling` (a per-level flag, cleared when a level is created; the client prints the text `nopkzone`

when its own attack is refused by it); - the attacker has not already been judged in the current second of the day: the routine keeps a per-attacker last-second stamp and drops a second judgement in the same second. The stamp is set before the guild and box tests; - the attacker's `(tag)` is not in the victim's nickname: the attacker's nickname is searched for its first `(`, then the first `)` after it with at least one character between them, and that bracketed text is searched in the victim's nickname (case-sensitive, anywhere). Nicknames are kept obfuscated in memory; - (swing path) the attacker's STATUS has neither bit 0x02 (hidden) nor bit 0x20, and the attacker is not in the kick animation (2.05 by animation name; 2.17.1 and later by an object id of 3). The hurt packet (40) handler checks the no-kill level flag, a per-player field that must be equal for attacker and victim, the 10-tile range, and caps the power at one heart.

Level flags seen

- `noplayerkilling` (script command) sets the no-kill flag.
- `sparringzone` (NPC script command) sets a second level flag that lets a player in the editor or paused state be hurt; the "can be hurt" check is: not in hurt mode, not dead, not swimming, hearts above 0, and (normal play or that flag).

Open

See open-questions.tsv. Main ones: the per-player field in the hurt-packet path; bit 0x20's meaning; knockback of the sword path; which other callers (bombs, arrows, baddies) use the hurt routine with which power; whether the guild test also applies to the hurt packet; the gmap list's order and the timing of its update against the entry save; the end-of-respawn save; the entry rule in the 2.1 test builds; no lab confirmation.

`client-internals/decomp-findings-2026-10-25-e/README.md`

Decompile mining, batch e: file-transfer request lists and save rules, 2026-10-25 (Asia/Tokyo)

Derived facts only. No raw decompile text, no addresses, no generated function names, no code, no paths. The raw exports stay private. Nothing was pushed to Graal_analysis, no public server was contacted, the lab was not touched. The lab facts quoted are from the 2026-10-24 captures that are already summarised in earlier notes.

Scope: what the real clients keep, what they re-request, and when a local file counts as up to date (Seihai queue item 9). Read in the 2.05 rev 2, 2.17.1, 2.22 and 2.31 clients; the 2.00 client only for the list code. Checked against the 2.17, 2.22 and 2.31 login and gmap captures.

Files

file	content
01-request-lists.md	the three name lists, the window of four, which loaders ask for what, how replies settle names
02-save-rules.md	whether and where a download is written, the folder table, the time of the saved file, name lookup
03-open-points.md	what was not resolved and what would settle it
04-name-index-case-link.md	which copy of a duplicated name is used, the A–Z case fold, and the packet 40 link (a horse does not change it)
request-rules.tsv	the request-side rules, one per row, with builds and evidence
name-index-rules.tsv	the index, fold and packet-40 link, one row each
download-folders.tsv	the save-folder decision table per build

file	content
open-questions.tsv	unresolved points

Top facts

1. A client holds three name lists: wanted, update-asked, settled. A file the client lacks is asked for with `PLI_WANTFILE`; a file it has is asked about with `PLI_UPDATEFILE` carrying the file's last-write time in Unix seconds.
2. Four requests of each kind are on the wire at once. More names wait in order; a reply for a name among the first four releases the name now in fourth place. The two kinds count separately. Identical in 2.00, 2.05 rev 2, 2.17.1, 2.22 and 2.31, and seen in the lab (four and four before the first reply).
3. A local file is checked once per name per connection. Any reply (file, up to date, send failed) puts the name in the settled list; a name in that list is not asked about again.
4. The client never compares times itself. The server decides, and answers with the file, "up to date" (45, 2.1+) or "send failed" (30, which before 2.1 also means up to date).
5. Image and animation loaders: local file means use it and send the check; none means ask for it. Sound loader: ask only when missing. Levels use no file transfer.
6. Downloads are kept as cached streams and written to disk only when the `saveimages` option is on (default on). A file the client already has is replaced where it is. A new file goes to `sounds`, `maps` (2.17.1 and later) or a folder under `levels` chosen by the name.
7. The saved file gets the server's time (2.1+) so the next check carries it. A lab 2.22 run saved a downloaded gmap under `maps` with the server's time.
8. A name the server could not send is not asked for again on the same connection (lab, 2.22).

`client-internals/decomp-findings-2026-10-25-e/01-request-lists.md`

Request lists

Lists

Each client keeps three lists of file names: - A (wanted): names asked for with `PLI_WANTFILE` (23, name only). - B (update asked): names asked about with `PLI_UPDATEFILE` (34: GINT5 time, then name). - C (settled): every name a reply came back for.

Asking

- No local copy: if the name is not in A, append it; send the request when A now holds at most four names.
- A local copy: if the name is in neither B nor C, append it to B; send when B holds at most four. A name in C is skipped, so each local file is checked once per connection.
- The time sent is the last-write time of the local file, Unix seconds. Lab: it matched the file's mtime for the sword, shield and body images at login.
- Builds whose packet set has no id 34 (1.33 to 1.41) never check a local copy.

Replies

A reply (file, up to date, send failed, or the end of a large file) removes the name from A and B. If the removed position was among the first four and names are still waiting, the one now in fourth place goes out. Then the name is added to C. The pair A/B releases independently.

Who asks

- Image loaders and animation loaders: local file, use it and check it; no file, ask for it.
- Sound loader: ask only when missing. The lab shows no check for a sound that exists.
- Levels: no file transfer; board packets and the level modtime exchange, plus the persistent web-level cache.
- A gmap file: checked like an image when present (lab, 2.22 gmap runs); asked for when missing (lab).

Failed names

A name answered with "send failed" is not asked for again on the same connection (lab: a missing step sound is asked for once however often the player walks). What the client stores for it beyond that is not known.

Cross-build

The list code is the same in 2.00, 2.05 rev 2, 2.17.1, 2.22 and 2.31. In 2.05 rev 2 and 2.22 the request-side function bodies were hard to read in the export; the request rules for those two come from the matching reply-side code and the lab, and are marked inferred in the tsv.

`client-internals/decomp-findings-2026-10-25-e/02-save-rules.md`

Save rules

Whether a file is written

A received file is first held in memory as a cached stream. It is written to disk only when the `saveimages` option is on; the option lives in the client's config file and defaults to on.

Where

1. If the client already has a file under that name, it is replaced where it is. The lookup tries the name as a path from the game folder, then the client's own file-name index (name to path and time).
2. Otherwise by name, case-insensitive:
    - `.wav`: `sounds`.
    - from 2.17.1 on: `.gmap`, and `.txt`, `.gif`, `.png`, `.mng` with `map` in the name: `maps`.
    - everything else under `levels`, in a folder by name: `bomys` (name starts `bomy` or `b_`), `ganis` (`.gani`), `emoticons`, `hats` (`hat` plus a digit as the fourth character), `heads`, `bodies`, `swords`, `shields` (the name contains `head`, `body`, `sword`, `shield`), else `webgifs`.
3. 2.05 rev 2 has no `maps` rule: its downloads all go under `levels`. 2.1 to 2.16 are inferred to match the later builds (no build in that range was read).

Lab: a downloaded gmap on 2.22 was saved as `maps/<name>`; images and animations went to the `levels` folders above.

Time of the saved file

From 2.1 the file reply carries a GINT5 last-write time; the client sets it on the saved file. Before 2.1 the reply has no time, so the file's time is the time of the download.

Names

Compared without regard to case. The fold is plain A–Z only, and the index then compares the folded bytes exactly; see `04-name-index-case-link.md`. Lookups use the same normalisation as the loaders.

`client-internals/decomp-findings-2026-10-25-e/03-open-points.md`

Open points

- ~~Persistent level cache (`GWEBL001`): format already in the notes; its interaction with `PLI_LEVELMODTIME` was not read here.~~ **Closed.** The write is on **server** packet 39 (`PLO_LEVELMODTIME`, not a client `PLI_`): `../../formats/gwebl001-cache.md` and `../../protocol/packet-layouts.md` **[2.05]**. Client level entry still uses board packets plus that modtime exchange (see also request-lists: levels use no file-transfer ids).
- Login default checks: the lab shows a check for three body-part images and a gmap; the full set (and whether they share a frame) needs a capture with a full and an empty data folder.
- ~~Scan order of the file-name index when two folders hold the same name.~~ **Closed.** No folder is preferred. 2.22 keeps the newer last-write time (an equal time keeps the first file the walk met); 2.05 rev 2 keeps the first hit of a depth-first search. A copy in the current directory wins over either subfolder (the game folder in 2.22; `levels` in 2.05 rev 2). `04-name-index-case-link.md` **[bin]**.
- ~~Save folders for 1.39 and 2.00 (save routines not read), and for 2.1 to 2.16 (inferred).~~ **Closed for 1.39 and 2.00** in `../decomp-findings-2026-10-25-k/02-save-folders-by-build.md` **[bin]**. **Still open for 2.1 to 2.16.** A 2026-11-01 listing of the client archive still has no build between the 2.1 test builds and 2.22.
- ~~Whether builds before 2.1 re-send the check for a settled name. Their reply has no time, so "up to date" has no form there; inferred only.~~ **Closed** for re-send: settled-list C skips a second check on the same connection in 2.00 and 2.05 — 01-request-lists.md **[bin]**. (Before 2.1, "up to date" is folded into send-failed id 30; that reply form is already noted there.)
- ~~What an empty cached stream after "send failed" does.~~ **Closed.** Save routine returns at once for a missing or zero-length stream: nothing written, nothing deleted — `../decomp-findings-2026-10-25-k/02-save-folders-by-build.md` (The guard) **[bin]**.
- ~~Whether the case folding is plain ASCII.~~ **Closed.** Yes: only `A`–`Z` are folded, then the index compares the bytes exactly. The locale compare is not used for file names. Same note **[bin]**.

`client-internals/decomp-findings-2026-10-25-e/04-name-index-case-link.md`

File-name index, case fold, and the packet 40 link

Static read of the 2.05 rev 2 and 2.22 clients. No capture and no client was run. **[bin]**

Case fold

File names are folded with the plain A–Z fold: each byte from `A` to `Z` is turned into `a` to `z`, and every other byte is left as it is. Accented letters are not folded, so they do not match across case.

The file-name index stores that folded base name (the part after the last `\` or `:`). A lookup folds the same way and then compares the bytes exactly. The runtime also has a locale compare, used by ordinary sorted string lists; the file-name index does not use it.

The same A–Z fold is applied before the folder tests (`head`, `body`, `sword`, `shield`, `hat` plus a digit, the `.code` skip, and the `pms` / `profiles` / `weblevels` skip below). **[bin]**

Which copy is used

Folder names do not decide the winner. The same base name in two folders is decided as follows.

Both builds, before the index: if a file of that name exists in the current directory, that file is used.

- 2.22 sets the current directory to the game folder at startup, so a copy in the game folder itself beats either subfolder.
- 2.05 rev 2 sets the current directory to the game folder's `levels` directory, so a copy placed directly in `levels` beats `levels/bodies` and `levels/webgifs`. Opening a world can point the current directory at that world instead.

2.22. At startup the client walks the game folder and fills a name-to-path index.

- In each directory the files are recorded in the order the directory listing returns them, and only then are subdirectories entered, in that same order.
- A directory whose parent is the game folder and whose folded name is `pms`, `profiles` or `weblevels` is not entered. A file whose folded extension is `.code` is not recorded.
- The key is the folded base name. The stored path is kept with the last-write time from the directory listing. A later time replaces the path. An equal or earlier time leaves the path already stored, which is the one the walk met first.

2.05 rev 2. A full-folder indexer with the same skip rules is in the program (it would keep the first name and ignore a later duplicate), but no call site in this image references it. What runs is the lookup:

1. The name as a file in the current directory, as above.
2. Else a path remembered from an earlier lookup, if that file is still there.
3. Else a depth-first search, first of an extra root recorded when a world file is opened, then of the game folder. In a directory, a file of the wanted name is taken before any subdirectory is entered. Subdirectories are tried in directory-listing order, and the search stops at the first hit. The same three folders and `.code` files are skipped. The hit is remembered for the next lookup.

So for two different bytes under the same name, 2.22 draws the newer file, and 2.05 rev 2 draws the first file the search meets. **[bin]**

Packet 40 link

Packet 40 (and the same test on the longer hit form) is dropped unless one integer on the attacker's record equals the same integer on the victim. A new player is created with that integer 0, so two ordinary players pass.

The horse is a different slot. Setting or clearing the horse image does not write the integer. A player who only has a horse still carries 0 and still matches another ordinary player. **[bin]**

In 2.22 the integer is the layer/parent link:

- It stays 0 until a `layer` property is applied. The value is then the layer number minus one. If that character has a parent, the parent's link is added as well.
- A spawned follower copies the source's link, or one more than the source's link. That is the follower's own link, not the player's. The player's link is unchanged by the spawn.

2.05 rev 2 has the same equality test and the same zero at creation, and its horse image is likewise a different slot. It has no `layer` property string. **[bin]**

`client-internals/decomp-findings-2026-10-25-f/README.md`

Real GR-V1.03 levels: what two supplied files and a 3113-level archive show

Date: 2026-10-25. Derived facts only: counts, offsets and structure. No level content, names or paths.

Material: two single `GR-V1.03` levels, and an old private player-world archive with 3141 files named `.graal` (27 empty, 1 text `GLEVNW01` level under a `.graal` name, 3113 binary `GR-V1.03`, 2812 distinct by CRC-32). Every binary level was decoded with the rules in `formats/level-binary.md` and `formats/level-binary-client.md`, using a second implementation (Seihai's reader), and checked against `formats/grv1-decoder/grv1.py`.

Verdict on the existing notes

Note	Result
13-bit codes for `GR-V1.03`	Confirmed. 1998 of 3113 boards use a tile id of 2048 or more; 12 bits cannot carry them.
Board rules (flag bit, low-byte count, `0x100` double, runs up to 64)	Confirmed. All 3113 boards give exactly 4096 tiles; the next section starts on the next byte. 2091 files use double runs, 1809 a run of 64, none a longer one.
Section order and terminators	Confirmed (see 01).
Baddy section ends at first byte >= 0x80	Confirmed; `grv1.py`'s extra "or x = 0" is wrong (see 03).
Sign alphabet, one table, `byte - 32`	Consistent; 2381 signs decode, 498 use `#K(n)` escapes (see 04).
GR-V1.04 `HEIGHTS` block	No file has one; nothing to confirm.
Link arrival "number or `playerx`/`playery`"	Incomplete (see 02).

Files

- `01-section-shapes.md` section layout as seen, the no-sign-section ending, coordinate units.
- `02-link-arrival-expressions.md` arrivals written as `playerY+N`.
- `03-baddy-terminator.md` x = 0 baddies and the `grv1.py` rule.
- `04-sign-and-chest-details.md` sign escapes, chest sign index -1, out-of-range positions.
- `corpus-stats.tsv` aggregate counts.

`client-internals/decomp-findings-2026-10-25-f/01-section-shapes.md`

Section shapes as seen in real GR-V1.03 files

After the board bit stream (padded to a whole byte) the file is, in order:

1. Links: one text line each, ended by a line holding only `#`. In every file the link section is `n` link lines plus `#\n`; a level with no links is just `#\n`.
2. Baddies: records of three raw bytes (`x`, `y`, type) plus a verse line ending in `\n`; the section ends with `FF FF FF` plus `\n` (the terminator is an empty verse line). Verses are separators only (`\\` seen as two backslashes for empty verses).
3. NPCs: one line each (`x+32`, `y+32`, `image#script`, script line breaks as `0xA7`), ended by a line `#`.
4. Chests: four bytes each (`x+32`, `y+32`, `item+32`, `signindex+32`) plus `\n`, ended by a line `#`. A level without chests still has the bare `#\n`.

5. Signs: records until end of file. **When a level has no signs the file simply ends after the chest `#\n`:** no sign record and no empty line. 2305 of 3113 files have no signs; 2308 files end with `#\n`.

Nothing follows the signs in any file: no `HEIGHTS`, no unknown trailing bytes (0 bytes left over in all 3113).

All counts of whole-tile coordinates check out against the board: in one supplied level a link's two tiles hold the same tile id (a door), and in the other the chest sits on its own tile id. NPC and baddy bytes are single bytes and may lie outside 0..63 (165 NPCs: x from -32 to 223, y from -11 to 223; 3 baddies with y = 255).

NPC image field: 10735 `.gif`, 1164 `.png`, 46 `.mng`, 1 `.mid`, 1 `.mp3`, and 2913 empty (script-only NPCs).

`client-internals/decomp-findings-2026-10-25-f/02-link-arrival-expressions.md`

Link arrivals are not only numbers or playerx/playery

`level-binary.md` lists a link as `dest x y w h newx newy` and the notes allow `playerx` / `playery` for the arrival. In the archive the last two fields hold, over 18,710 coordinate fields:

Form	Count
plain number (integers and half tiles such as `24.5`)	12035
`playerx` / `playery`	6652
`playerx` or `playery` followed by a signed number (`playery+10`, `playerx-15`, `playery-1.5`, `playerx+15`)	21 (in 14 levels)
a stray non-numeric word (one level wrote two such words)	2

Open: whether the stock client evaluates the offset, reads it as a number (0), or takes only the keyword. No capture shows it. The two level names that end in neither `.graal` nor `.nw` (`.enigma` 10, none 2) are also unexplained.

Link destination extensions: `.graal` 9203, `.nw` 140, `.enigma` 10, none 2.

`client-internals/decomp-findings-2026-10-25-f/03-baddy-terminator.md`

Baddy section: only a first byte >= 0x80 ends it

Two levels have a baddy at x = 0 (one at (0, 0), one at (0, 62) in a row of 16 baddies spaced 4 tiles apart ending with x = 60). `level-binary-client.md` says the section ends at a first byte of 0x80 or more; that is right. The reference decoder `formats/grv1-decoder/grv1.py` also stops at x = 0, which loses the x = 0 baddy and any after it (3 baddies in these two files) and then reads the rest of the baddy bytes as NPC lines, so NPC counts are wrong in those two files (26 against 25, 12 against 10 on the NPC count).

Corpus baddy totals: 902 baddies in 99 levels; types 0 to 9 all occur. A record's three bytes can be 0x0A or 0x00; the reader must look for the line end only after the first three bytes.

client-internals/decomp-findings-2026-10-25-f/04-sign-and-chest-details.md

Signs and chests in the corpus

- 2381 signs in 808 levels; 684 sit at (0, 0) (dialogue text kept by index, as `level-binary-client.md` guessed).
- 498 `#K(n)` escapes with n not 13, the commonest n values being 95 (_), 164, 61 (=), 42 (*), 36 ($), 64 (@), 94 (^), 124 (|), 134, 233, 169, 43 (+). Characters missing from the 97-entry table are written this way, as `sign-text.md` says.
- 375 chests in 88 levels. Item indexes seen: 0 to 11, 16, 17, 19, 24, and one 78 (outside the 25-name table). 343 of the chests have sign index -1 (byte `0x1F`), which is "no sign".
- The old Seihai decoder (`char + 89`) lost punctuation and line breaks; it is replaced by the alphabet decoder.

client-internals/decomp-findings-2026-10-25-g/README.md

Decompile mining, batch g: ids 24, 46 and 178, 2026-10-25 (Asia/Tokyo)

Derived facts only. No raw decompile text, no addresses, no generated function names, no code, no paths. The raw exports stay private. Nothing was pushed from here to Graal_analysis directly, no public server was contacted, the lab was not touched. The lab facts quoted are from the 2026-10-24 captures that are already summarised in earlier notes.

Scope: the three fields earlier notes called unknown (Seihai queue item 12): the meaning of client id 24 (relayed as 32), the meaning of client id 46, and how the clients read server id 178. Read in the 2.17.1, 2.22 and 2.31 clients (all three read for 24 and 178; 46 in 2.31, with a negative check in 2.17.1 and 2.22), plus the GServer-v2 and graal-serverlist sources.

Files

file	content
01-id24-image-list.md	what id 24 and its relay carry: the weapon-script image list, the full-list marker, the sender's rules, the receiver's rules
02-id46-level-switch.md	where id 46 is sent and why its payload is a literal
03-id178-comma-text.md	how the clients split the 178 text, what they do with each field, what the writers send
04-open-points.md	what was not resolved and what would settle it
image-text-format.tsv	the image-record text, one row per tag
comma-text-rules.tsv	the clients' comma-text splitter, one rule per row
open-questions.tsv	unresolved points

Top facts

1. Id 24 is a list of the images that the sender's weapon scripts show (`showimg` and the `changeimg*` commands), not a heartbeat and not an emote. The one byte 9 that real 2.22 and 2.31 clients send is the list's first character on its own: a "full list follows" marker with no images after it.
2. The sender keeps the list it sent last and sends only changes, except that every 101st call of a periodic handler it sends the whole list, led by the marker. The call after a send is skipped, so a wrap is 102 ticks:

the lab's 5.1 s if a tick is 50 ms (inferred from the period). With no images and no changes the only message is the marker.
3. The receiver (applies only for a player in its active level) creates an image when it gets a name, removes it on an empty name, changes the other fields only of an image that exists, and after a marked text removes every image the text did not name.
4. The 2.22 client sends its TCP copy only when UDP does not reach some player in the level; with nobody in the level that test is false. In 2.31 the test is a constant true. This explains the lab (2.22 only with a second player, 2.31 always). 2.17.1 has the same sender and test.
5. Id 46 is a content-free notice. One send site in 2.31 (none in 2.17.1 or 2.22): the start of the routine that switches the client's current level, when connected. The payload is a literal 1. GServer-v2 ignores it.
6. Id 178 is read by the clients' own comma-text reader, the same in all three builds: leading white space skipped, quotes with doubled quotes, an unquoted item ends at white space or a comma, at least 4 items, items 1 to 3 used. A name with a space must be quoted or it shifts the later items.
7. Item 1 of 178 is kept as the server's name: the first two characters are dropped when the second is a space (a listserver type prefix such as `P Name`). 2.31 shows it in the window title. It is not used to connect.
8. What a real list service sends for 178 has still never been captured. The open-source reimplementation sends four quoted items; a commented-out older line in its source shows a different shape with a tag in item 0.

`client-internals/decomp-findings-2026-10-25-g/01-id24-image-list.md`

Id 24 (`PLI_SHOWIMGPLAYER`) and its relay (`PLO_SHOWIMG`, 32)

Read in the 2.17.1, 2.22 and 2.31 clients. The sender routine is the same in all three (same structure, same constants); the receiver was read in 2.31.

What the text is

A list of the images that the sender's weapon scripts currently show (`showimg` and the `changeimg*` commands), one record per image. A record starts with a byte that is 0x2a plus the image's index (0 to 199). Fields follow, each a tag byte (0x20 plus the tag number) and its data. See `image-text-format.tsv`.

A tag byte of 0x2a or more starts the next record, so tags run 0 to 8. An unknown tag skips its one data byte. A tag byte with no data byte after it ends the whole text.

The first byte of a text may be the marker 0x29 (a closing parenthesis). As a GCHAR that is the value 9, which is the one byte every real client was seen to send. It means "this is the full list"; here the list is empty.

Sender

- It collects the images of the player's weapons (index 0 to 199) and compares them with the list it sent last. Changed images are sent with the fields that changed; an image that is gone is sent with an empty name.
- A counter runs from 100 down; at a wrap the text starts with the marker and every image goes out in full. The wrap is the 101st call of a periodic handler. The call after a send is skipped, so a wrap is 102 ticks. The lab period of about 5.1 s fits a tick of 50 ms; that interval was inferred from the period, not read from the client's form resource.
- Nothing is sent when the text is empty. With no images the only text that ever exists is the marker alone at a wrap, one byte: GCHAR 9. The first tick after login is a wrap.

- The client sends a UDP copy when it can (the 2.05 UDP id 15 form for older builds). The TCP copy goes out only when its test says so.

The TCP test

- 2.31: the test is a constant true. The TCP copy goes out at every wrap, alone or not, in any level.
- 2.22 and 2.17.1: the same test in both. It is true when UDP does not reach some player in the client's level; with nobody else in the level there is nothing to find and it is false. This fits the lab: 2.22 sent only with a second player present (no sends in three solo sessions), and the 90 s solo 2.17 capture saw no id 24.
- A wrap that finds the test false is lost. The next wrap is a full period later.

Receiver

- The relay is `GSHORT sender id`, then the sender's text unchanged (GServer-v2 does not look inside it).
- The 2.31 handler reads the sender's id, finds that player in the client's active level and hands the rest to the routine that applies the text. Nothing else reads the byte.
- Per record the image is looked up by index. Name: an empty name removes the image; a non-empty name creates it if missing (defaults: layer 1, colours 200 each, zoom 10, mode 0) and sets the name. Other fields are applied only to an image that exists.
- Part: 0x20 means the whole image. 0x21 is followed by left and top as two 7-bit bytes each, then width and height as one byte each; the values are clamped (left and top 0 to 16000, width and height 0 to 220).
- After a marked text, every image of that player that the text did not name is removed.

What is not known

- The sender's own value formatting for each field. The units above are the receiver's reading, so the units of x, y, colours and zoom are unconfirmed.
- What the per-player "UDP reaches this player" flag in the 2.22 test is and how it is set.
- How the client draws the images (layers, the screen-unit mode of layer 10 and over, colour and zoom).
- 2.05 uses the same tagged form (tags 0 to 6) with a 60-call full-list countdown: `../id24-205.md`. Builds before 2.05 were not read.

`client-internals/decomp-findings-2026-10-25-g/02-id46-level-switch.md`

Id 46 (`PLI_ENTERLEVEL`)

Read in the 2.31 client; 2.17.1 and 2.22 were checked for a send site and have none.

- One send site. It is at the start of the routine that switches the client's current level, whenever the client is connected, before anything else the routine does. This fits the lab: one 46 at login, one per level change, and in a gmap one per edge crossing, always before the props frame that carries the new cell prop.
- The payload is the one-character text GCHAR 1. It is a constant in the code. No level, player count, connection or option value goes into it. GServer-v2's comment guessing "a player count for the gmap level" is not what the client sends.
- The server side in GServer-v2 is a stub that does nothing.
- 2.17.1 and 2.22 have no send site for id 46. This agrees with the lab (neither sent it) and with the version profiles (46 is added in the 2.3 step).

Not resolved: what a real service does with it. Stock GServer-v2 does nothing, so no capture against it can show a reaction.

`client-internals/decomp-findings-2026-10-25-g/03-id178-comma-text.md`

Id 178 (`PLO_SERVERWARP`)

Read in the 2.17.1, 2.22 and 2.31 clients (the handler, the text splitter and the connect routine were compared and match in all three).

The handler

Build a string list from the text with the client's comma-text setter, require at least 4 items, and call the "connect to another server" routine with items 1, 2 and 3. Item 0 is not read. Anything shorter does nothing (lab: no reaction, connection stays open).

The splitter

The same code as the standard list class of the toolchain the clients are built with. Rules in `comma-text-rules.tsv`. The ones that matter for 178:

- White space (bytes 1 to 0x20) before an item is skipped.
- A quoted item ends at its closing quote; two quotes in a row are one quote; an unclosed quote runs to the end of the text.
- An unquoted item ends at a byte of 0x20 or less or at a comma. A space inside an unquoted name therefore ends the item and the rest of the name starts the next one.
- After an item: white space, then one comma, then white space. A comma at the very end adds no empty item; an empty item between two commas is kept.
- A NUL ends the text.

Consequence: a server name that contains a space must be quoted by the sender. `x,Main Server,h,1` reads as five items and the client would connect to the host `Server`.

The connect routine

- Item 1 is the server name. If its second character is a space the first two characters are dropped (type prefix, as in `P Name`). The result is stored as the current server's name. In 2.31 it is also put in the window caption: the caption is the name alone if the name already contains "graal" (any case), otherwise `Graal:` plus the name.
- Items 2 and 3 are stored as the host and the port, both as text first.
- The routine then reconnects to the host and port. The name is not used to connect, which agrees with the lab (a made-up name connected normally; reordered host and port failed to connect).

What the writers send

- GServer-v2: forwards the listserver's answer to `PLI_SERVERWARP` (41) as 178, unchanged, and only to clients from 2.1 on. The server's own code calls this a hack to let the new clients serverwarp.
- graal-serverlist: for a request it answers with the server name twice, the host and the port, as one comma text made by its tokenizer (each item quoted, quotes doubled). A commented-out older line in the same

function builds `playerworld` plus a number, then the name in quotes, then the host and the port: item 0 is a tag of that kind there. That is source-comment evidence only.
- A real service's text has never been captured.

Seihai reads the text with the same rules (`delphi_comma_text`) and shows the name without the type prefix.

`client-internals/decomp-findings-2026-10-25-g/04-open-points.md`

Open points, batch g

See `open-questions.tsv` for the list with what would settle each. In short:

- Id 24: the sender's own field formatting (the layout above is the receiver's); a capture of a real client whose weapon shows an image would settle it. The lab cannot do this yet because the real client did not run NPC scripts there. The per-player "UDP reaches this player" flag of the 2.22 test. Drawing rules for layers. The 2.05 form is settled in `../id24-205.md`.
- Id 46: only a service that reacts to it could tell what it is for; stock GServer-v2 ignores it.
- Id 178: the real service's text (item 0 in particular), and whether the port is converted with a range check.

`client-internals/decomp-findings-2026-10-25-j/README.md`

Z3-V1 (`.zelda`) levels: format checked against 139 real files and three clients

Date: 2026-10-25. Derived facts only: counts, offsets and structure. No level content, names, paths or code addresses.

Material: the old client builds' level folders (1st Graal to 15th Graal, Graal 1.1 and 1.1.9, the 2nd/4th "alpha" folders, and the Java Zelda Online game folder with its source), which hold 663 `Z3-V1` files (139 distinct by content). Each file was decoded twice, with a throwaway Python scanner built on `formats/grv1-decoder/grv1.py` and with Seihai's new strict Rust reader, and the two agreed. The Java Zelda Online source, the 1st Graal client and the 1.39 client were read for the loader rules (derived facts only; no decompile text is kept here). A private player-world archive was counted for comparison.

Verdict on the existing notes

Note	Result
Sections: board, links, baddies, signs; no NPC and no chest section	Confirmed. All 139 distinct files end exactly at the end of the signs.
12-bit codes for `Z3-V1.03` / `.04`	Confirmed (all 139 boards give 4096 tiles and the links start on the next byte).
Verse line only in `Z3-V1.04`	Confirmed, with detail: see 01. `Z3-V1.03` records have an empty rest-of-line, so every record is 4 bytes (3 + line end), as in the Java source.
Baddy terminator `FF FF FF` + line end	Confirmed.
Links `dest x y w h newx newy`	Incomplete: `Z3-V1.03` writes decimals and `-1` arrivals (see 02).
`Z3-V1.02`	Only the Java client reads it; no real file has it.
"Only GR-V1.00 to 1.03 load, `.zelda` Z3-V1 for the offline levels" (1.33 to 1.39)	Incomplete. Every Graal client binary from 1.28b2 to 2.31 carries both `Z3-V1.03` and `Z3-V1.04` magics, and the 1st Graal and 1.39 loaders accept them whatever the file name.

Files

- `01-record-shapes.md` board, link, computer and sign records as seen; computer types; `Z3-V1.04` verses.
- `02-links-and-arrivals.md` decimal links, negative arrivals, destination extensions.
- `03-extensions-and-magic.md` which names hold which magic; what the loaders key on.
- `04-loader-rules-compared.md` Java, 1st Graal and 1.39 rules side by side, with the differences that matter.
- `05-old-reader-failures.md` what a `.graal`-style section reader gets wrong on `Z3-V1` (counts).
- `by-build.tsv` per-build counts (18 builds).
- `format-stats.tsv` aggregate counts over the 139 distinct files.

Open

- (Answered later in `../bare-level-names.md`: no build tries `.zelda` for a bare name and none falls back; the editor adds `.graal` to a name saved without an extension.)
- The 2.0 to 2.31 loaders were checked for the magic strings only, not traced.
- Java sign alphabet versus the shared one for codes 82 to 88 (5 real signs touch 82 or 83; not compared).
- Computer type names beyond the Java source's 0 to 6 (none seen).
- No `Z3-V1.02` sample.

`client-internals/decomp-findings-2026-10-25-j/01-record-shapes.md`

Record shapes seen in the real files

Counts are over the 139 distinct files (85 `Z3-V1.03`, 54 `Z3-V1.04`); see `format-stats.tsv`.

Layout

```
char[8]  magic      Z3-V1.03 | Z3-V1.04
bits     board      4096 tiles, 12-bit codes, run rules of GR-V1.00 (formats/level-binary.md)
text     links      `dest x y w h newx newy` lines, then a line holding `#`
binary   computers  x y type (3 raw bytes) + rest of line; ends at a first byte >= 0x80
text     signs      `x+32 y+32 text` lines until an empty line or the end of the file
```

All 139 files end with a line end. Sizes run 443 to 5369 bytes (median 1919).

Computers ("baddies")

- Zelda Online calls them computers. A record is `x`, `y`, `type`, then the rest of the line and a line end. `x` and `y` are raw tile bytes (not the +32 of signs); no coordinate is 64 or more.
- `Z3-V1.03`: the rest of the line is empty in all 124 records, so a record is 4 bytes.
- `Z3-V1.04`: the rest of the line holds three verses separated by backslashes (sight, hurt, attack). 45 of the 45 records carry the verse form, 40 of them with all three verses empty (two backslashes only).
- Types are 0 to 6 in every file. Counts over all records: type 0 29, 1 29, 2 30, 3 23, 4 33, 5 5, 6 20. The Java source names them: green soldier, blue soldier, red soldier, shot soldier, swamp soldier, jumping hare, octopus. Later builds' names for the numbers are not known.
- Two records (both `Z3-V1.03`) have `0x0A` as x or y. They are data, not a line end: a reader that finds the line end before taking three bytes ends the section there.

- The section ends at `FF FF FF` and a line end (the first byte is the test: any value of 0x80 or more). 42 files have computers, 97 have none and go straight to the terminator.

Signs

- 120 signs in 63 files. `x+32`, `y+32`, then text in the shared packed alphabet (`formats/sign-text.md`). No empty sign text.
- 86 signs contain a byte of 0x80 or more where a line break goes (as in `GR-V1`).
- 5 signs contain alphabet code 82 or 83; the Java source reads codes 82 to 88 differently (see README, open items).
- The sign list ends at an empty line or at the end of the file; every real file ends at a line end after the last sign.

`client-internals/decomp-findings-2026-10-25-j/02-links-and-arrivals.md`

Links: decimals, negative arrivals, destination names

Counts over the 139 distinct files: 338 links in 136 files (`Z3-V1.03` 207, `Z3-V1.04` 131).

Decimals

- `Z3-V1.03` links are written with decimal points: 167 of 207 have a decimal arrival, 124 have a decimal x, y, width or height (47 of the 85 `Z3-V1.03` files). Geometry is whole tiles with a `.0` ending; arrivals are often half tiles (171 links have a fractional arrival, 79 in `1.03`, 92 in `1.04`).
- `Z3-V1.04` links have no decimal geometry (0 of 131) but 92 have a decimal arrival.
- All three loaders read six floating-point numbers (the Java source with `parseDouble`-style reads, 1st Graal and 1.39 with string-to-float). A reader that parses integers drops these links.
- No destination name contains a space (0 of 338), so splitting the first token off is safe for these files. `formats/level-binary.md` says tokens before the last 6 are rejoined for GR-V1; the Delphi clients take the destination up to the first space.

Negative arrivals

- 67 links have a negative x arrival and 85 a negative y arrival (151 links in all; one has both). `Z3-V1.03` and `.04` both use them.
- Rule, from the Java source and 1st Graal: a negative arrival coordinate is a flag meaning "keep the player's own coordinate on that axis". Both set a "constant" flag from `< 0` and use the player's position otherwise.
- The same appears in `GR-V1` links: 805 of 5381 distinct links in the same builds have a negative arrival. That the 2.x clients read it the same way is inferred, not traced.

Destination extensions

- `Z3-V1.03` links name `.zelda` (205) or `.editor` (2) files. `Z3-V1.04` links name `.graal` (76) or `.zelda` (54), or `.editor` (1).
- The extension in a link therefore differs between builds that name their levels `.graal` and builds that name them `.zelda`; the loader sees the magic only (see 03).

`client-internals/decomp-findings-2026-10-25-j/03-extensions-and-magic.md`

Extensions say nothing; the magic does

Distinct files (by content) in the old builds, by magic and extension:

Magic	`.zelda`	`.graal`	`.editor`	other
`Z3-V1.03`	84	0	1	0
`Z3-V1.04`	23	31	0	0
`GR-V1.00` to `GR-V1.03`	694	1271	9	111 (assorted suffixes)

- The `.zelda` extension is not a format marker: 694 of 801 distinct `.zelda`-named files are `GR-V1.00` to `GR-V1.03`. 2nd Graal to 15th Graal ship 68 to 368 such files per build (`by-build.tsv`, column `gr_magic_in_.zelda`).
- 1st Graal's 16 levels are all `Z3-V1.04` under `.graal` names.
- The Java Zelda Online game folder holds 51 `Z3-V1.03` files, all `.zelda`, and no `Z3-V1.04`.
- The player-world archive (`.graal` and `.nw` names) holds no `.zelda` file and no `Z3-V1` file.
- Every client binary tested (1st Graal, 1.28b2 to 1.41, 2.0 to 2.31, the two NewWorld betas) carries both `Z3-V1.03` and `Z3-V1.04` as magic strings; none carries a third `Z3-V1` revision. The 1st Graal and 1.39 loaders were read: they compare the 8 magic bytes only. 2.0 to 2.31 were not traced beyond the strings.

`client-internals/decomp-findings-2026-10-25-j/04-loader-rules-compared.md`

Loader rules compared

Three readers were compared for the points where a re-implementation can go wrong. "Delphi clients" means 1st Graal and 1.39, which agree on every row unless stated.

Point	Java Zelda Online	Delphi clients	Real files
Code width	11 bits for `Z3-V1.02`, 12 for `.03`	12 for `Z3-V1.03/.04`	12 (no `.02`)
Control code	top bit set, low byte count, `0x100` double	same	same, 4096 tiles in all
Double flag	stays set until a double run completes	not separated	the `GR-V1` reader clears it at the next control code; no file shows a difference (0 of 663)
Link numbers	six doubles	six floats	decimals present
Negative arrival	"constant" flag from `< 0`	same	151 links
Computer record	4 fixed bytes read as x, y, type, line end (`Z3-V1.03` form)	1.39: bytes are added until a line end after 3 or more bytes; a first byte below 0x80 starts a record. 1st Graal reads by line (inferred from its shape)	all `.03` records have an empty rest
`0x0A` as x or y	harmless (fixed bytes)	harmless in the 1.39 loader (it keeps adding bytes until the line is longer than 2); 1st Graal would end the section	2 records (not run in 1st Graal)
Verses	none	split at backslashes, three strings (`.04`)	`.04` only
Signs	`x+32`, `y+32`	same, text `byte - 32` through the shared alphabet	consistent
Sign list end	empty line or end of file	empty line	both occur

Consequences for a re-implementation:

- Read the three computer bytes first, then the line; do not look for the line end before taking x, y, type.
- Parse link numbers as floating point and treat a negative arrival as "keep the player's coordinate".

- Do not dispatch on the extension.
- The double flag rule and the sign-end rule do not matter for any real file; follow the clients.

`client-internals/decomp-findings-2026-10-25-j/05-old-reader-failures.md`

What a GR-V1 section reader gets wrong on Z3-V1

A section reader written for `GR-V1.03` (integer links, three-byte baddy records, an NPC and a chest section where `Z3-V1` has none) was replayed over the 139 distinct files. Counts:

Fault	Effect	Count
Link numbers parsed as integers	decimal links dropped	124 links in 47 `Z3-V1.03` files
Computer line end left in the stream	next record starts one byte late; the type byte of the record and the line end shift	all 31 `Z3-V1.03` files that have computers (one file: 14 computers reported, 10 present)
Negative arrival read as a literal coordinate	arrival at -1	151 links (also 805 `GR-V1` links)
`.zelda` extension treated as a promise	`GR-V1` files read as `Z3-V1`	694 distinct files under `.zelda` names are `GR-V1`

After switching to a strict reader every one of the 663 copies parses to exactly the end of the file with no leftover bytes and no anomaly.

`client-internals/decomp-findings-2026-10-25-k/README.md`

Decompile findings, batch k: hit knockback, packet-hit gates, save-folder layouts by build, 2026-10-25 (Asia/Tokyo)

Derived facts only. No raw decompile text, no addresses, no generated function names, no code, no paths. The raw exports stay private. No public server was contacted and no lab run was made for these facts.

Scope: the leftovers of batch d (damage and respawn) and batch e (file transfer). Static read of the 2.05 rev 2, 2.17.1, 2.22 and 2.31 clients (hit path), and of the 1.33 to 1.41, 2.00 to 2.05, both 2.1 test builds, 2.17.1, 2.22 and 2.31 clients (save routine).

Files

file	content
01-knockback-and-packet-hits.md	the knockback of a hit, the gates of packets 40 and 46, the 46 box, what stays unknown
02-save-folders-by-build.md	the save routine's folder layout in each build, the empty-stream guard
03-open-points.md	what was not resolved and which capture would settle it
knockback-probes.tsv	the body-box probe offsets
save-folders-by-build.tsv	the folder layouts per build
open-questions.tsv	unresolved points

Top facts

1. A hit starts hurt mode (40 ticks) and moves the player one tile per tick for the first five ticks along the knockback vector scaled to length one (a zero vector means east). After each step twelve body-box points are tested; if one is blocked the step is undone. Same in 2.05 rev 2, 2.17.1, 2.22, 2.31; the twelve offsets are identical in all four.

2. Packets 40 and 46 go through the sword's gates (attacker known, victim hurtable, AP not 100, level not no-player-killing, the once-a-second stamp per attacker shared with the sword and taken before the guild test, the guild tag test) and then a range test (40) or a box test (46). There is no attacker status test on the packet path. Power is capped at one heart; power 0 is accepted.
3. Packet 46's hit point is (b1/2, b2/2); a player's centre is (X+1.5, Y+2.0); the box is strictly under 2 tiles on each axis. The knockback is centre minus hit point.
4. The integer compared in packet 40 starts at 0. A horse does not change it, so a player who only has a horse still matches an ordinary player. In 2.22 it is the layer/parent link. See `../decomp-findings-2026-10-25-e/04-name-index-case-link.md`.
5. Save folders: 1.33 to 1.41 save everything in one folder (`levels/webgifs`); 2.00 and 2.01 have the folder list without `hats`; `hats` appears in 2.02; both 2.1 test builds have no `maps` folder rule, which first shows in 2.17.1.
6. Every build from 1.39 to 2.31 returns from the save routine at once for a missing or zero-length stream: nothing written, nothing deleted.

`client-internals/decomp-findings-2026-10-25-k/01-knockback-and-packet-hits.md`

Knockback and the packet hits

Knockback of a hit

The routine that hurts the player takes a power in hearts and a knockback vector (dx, dy). It sets hurt mode (mode 3, counter 40) and stores the vector scaled to length one tile. A zero vector is replaced by one tile east.

For the first five ticks of hurt mode (the counter reads 39 down to 35 after its decrement) the player moves one tile along the stored vector each tick, then the body box is tested at twelve points; if any is blocked the step is undone and the push is tried again the next tick. On open ground the player ends five tiles away after 250 ms. The test uses the same ground rules as an ordinary walk (blocking tiles, and chests).

The twelve points lie on the edges of a box 0.6 to 2.4 tiles across and 1.1 to 2.9 tiles down from (X, Y) (the position properties): three points per edge, the four corners counted twice; see `knockback-probes.tsv`. The box is centred on (X+1.5, Y+2.0). The routine and the table are the same in 2.05 rev 2, 2.17.1, 2.22 and 2.31.

Where the vector comes from: - sword hit: from the attacker's position to the victim's; - packet 40: its two bytes (the byte minus 0x60, over 16, tiles); - packet 46: the victim's centre minus the hit point.

Gates shared by the sword and the packets

Read in 2.05 rev 2 and 2.22. In this order: 1. the attacker's player id is known; 2. the victim is hurtable (not already hurt, not dead, not swimming); 3. the victim's AP is not 100; 4. the level has not run no-player-killing; 5. the attacker's once-a-second stamp: the second of the day is compared with the one stored for that attacker and stored; the table is shared by the sword and both packets, and the store happens before the guild test; 6. the guild test: the first bracket pair in the attacker's nickname must not occur in the victim's nickname; 7. for the packets, the per-player field test (below); 8. packet 40: the squared distance between the two X/Y positions is at most 100 (ten tiles). Packet 46: the box test below.

The sword path adds the reach box and an attacker status test; the packet path has **no** attacker status test.

Power: packet power is capped at one heart; a power of 0 is accepted (hurt mode and knockback, no hearts taken).

Packet 46 box

The hit point is (b1/2, b2/2) in tiles, from the two bytes after the power. The player's centre is (X+1.5, Y+2.0). A hit lands when the absolute difference is strictly under 2 on each axis.

The per-player field of packet 40

Packet 40 is dropped unless a link integer on the attacker equals the same integer on the victim. A new player is created with 0. The horse image is a different slot and does not write the integer, so a player who only has a horse still matches an ordinary player. In 2.22 the integer is the layer/parent link (layer number minus one, plus the parent's link when there is a parent). Detail: `../decomp-findings-2026-10-25-e/04-name-index-case-link.md` **[bin]**.

What stays unknown

- ~~Which object kinds (bomb, arrow, baddy) call the hurt routine with which power~~ — closed in `../damage-sources.md`.
- ~~Whether the respawn routine's second entry save lands on the death spot online~~ — closed in `../lab-captures-2026-10-25/01-damage-chest-respawn-weapons.md` §D (stays the entry point).
- ~~Whether the real client sends CURPOWER after damage~~ — closed in the same lab note §C.
- ~~Packet 40's per-player owner/parent field with a horse or similar (still needs a capture).~~ Closed in `../decomp-findings-2026-10-25-e/04-name-index-case-link.md`: a horse does not change the link.

`client-internals/decomp-findings-2026-10-25-k/02-save-folders-by-build.md`

Save-folder layouts by build, and the empty-stream guard

Each build's save routine for a downloaded file was read (static). The table is `save-folders-by-build.tsv`.

Layouts

- **1.33 to 1.41** (1.33, 1.34, 1.35, 1.36, 1.37, 1.38, 1.39, 1.41 read): one folder for everything that is saved, `levels/webgifs`. The 1.39 routine is entered only for names ending `.gif`, `.png` or `.mng`, so no sound is saved. It deletes an existing file with the same path first, writes the stream, and sets no modification time.
- **1.42 NewMain**: the two builds differ. The earlier one adds `ganis`, `heads`, `swords`, `shields`; the later one adds `sounds` (for `.wav`) and `bomys`. Not read further.
- **2.00 (all four revisions) and 2.01 (both)**: `sounds` for `.wav`. Under `levels`: `bomys` for names starting `bomy` or `b_`, `ganis` for `.gani`, `emoticons` for names starting `emoticon`, `heads`, `bodies`, `swords`, `shields` for names containing those words, else `webgifs`. There is no `hats` folder: a `hat0.png` goes to `webgifs`. There is no `maps` folder.
- **2.02 (all three revisions), 2.03, 2.04, 2.05 rev 1 and 2, both 2.1 test builds**: the same list plus `hats` (name starts `hat`, fourth character a digit, in the builds that test it). Still no `maps`.
- **2.17.1, 2.22, 2.31**: the same, plus `maps` for `.gmap` and for `.txt`, `.gif`, `.png`, `.mng` names containing `map`.
- 2.1 to 2.16 have no binary: when `maps` arrived between the 2.1 test builds and 2.17.1 is unknown.

The guard

Every build read (1.39, 2.00, 2.05 rev 2, 2.17.1, 2.22, 2.31) begins the save routine with: the stream exists and its size is above zero. Otherwise the routine returns: nothing is written, an existing copy is not deleted, the

modification time is not touched and the file is not announced to the loaders. A stream left empty by a failed send is therefore never saved.

Login request set (compared with traces)

In the 2.22 and 2.31 lab traces (real client against a real server, and Seihai in place of the client, same server) the login requests of the two clients match for the body-part images, icons and the gmap. The remaining difference is the second default-weapon image, which the real clients ask for and Seihai did not: the real order is the bomb image, then the bow image.

Evidence about the default weapons (not used to wire anything): the open server source numbers the default items with bow 7 and bomb 8; the 2.22 client's table of item names follows the same order; the real login asks the bomb image before the bow image, as a walk over default weapons 8 then 7 would. Which wire id names which image was not seen directly.

`client-internals/decomp-findings-2026-10-25-k/03-open-points.md`

Open points

- ~~Bomb, arrow and baddy hurt powers: the object-kind mapping was not traced.~~ **Closed.** Powers and kinds are in `../damage-sources.md` **[bin]** (explosion types 0.5/0.5/1, arrow 0.5, fireball 1, baddy touch 0.5 / golden warrior 1, etc.). Lab bomb/arrow blast-shape and arrow type-byte quirks remain open in `../lab-captures-2026-10-25/01-damage-chest-respawn-weapons.md` and do not reopen the power table.
- ~~The per-player field of packet 40: inferred to be always equal for ordinary players. A real client hit by packet 40 from a player that owns a horse or similar would show whether it differs.~~ **Closed.** The integer starts at 0 and the horse image does not write it, so a player who only has a horse still matches an ordinary player. In 2.22 the integer is the layer/parent link. `../decomp-findings-2026-10-25-e/04-name-index-case-link.md` **[bin]**.
- ~~End of respawn: the respawn routine calls the entry save once more after asking for the warp; whether the next respawn point is then the death spot online needs two respawns on a server that delays the level reply.~~ **Closed.** Lab: next respawn stays the level entry point, with or without a delayed reply — `../lab-captures-2026-10-25/01-damage-chest-respawn-weapons.md` §D **[cap]**.
- ~~CURPOWER after damage: needs a real client at full hearts hit by an injected packet 40 and a look for a props message with the hearts byte.~~ **Closed.** After a hit the client sends one props message with CURPOWER (half-hearts) + `hurt` GANI — same lab note §C **[cap]**.
- ~~Default weapon ids against images: see 02.~~ **Partly closed.** Lab §E: id **7** → `wbowi1.png` settled; id **8** → `wbomb1.png` not isolated (bomb image is asked anyway). Also ids 12, 20–23 → named images. Same lab note **[cap]**.
- ~~Level cache format (GWEBL001): not part of this batch.~~ **Closed** for format and key handling: `../../formats/gwebl001-cache.md` **[2.05][test]** (144 real files). Write is triggered by server packet 39 (`PLO_LEVELMODTIME`).
- ~~File-index scan order for two files with the same name in different folders: a capture with different bytes in `levels/webgifs` and `levels/bodies` and a look at which is drawn.~~ **Closed.** Neither folder wins by rule. 2.22 keeps the newer file; 2.05 rev 2 keeps the first hit. Same note **[bin]**.
- ~~Whether pre-2.1 clients re-send the file check for a settled name: a 2.05 run against a server that answers every check with "send failed".~~ **Closed.** Settled-list rule: any reply puts the name in C; a name in C is not asked again. Same list code in 2.00 and 2.05 — `../decomp-findings-2026-10-25-e/01-request-lists.md` **[bin]**.

- When the maps folder rule arrived (2.1 to 2.16): still no binary. A 2026-11-01 listing of the client archive has the two 2.1 test builds (already read: no `maps` rule) and then 2.22, and nothing between.

`client-internals/decomp-findings-2026-10-25-l/README.md`

Decomp findings 2026-10-25 (l): walking body box, arrays.dat, cut/lift, a public-server run

Derived facts from reading the 2.05 client (walking, lifting, sword cut, chest open) and from one login to a public GServer-RS with a 2.22 profile. No addresses, no function names, no decompiler output, no code. Numbers are tile units unless stated.

- `01-walking-body-box.md` — the wall test of the walking routine: a box with four probe points per facing, keys applied one at a time, corner nudge. Corrects `../foot-test.md`'s "official ~2.05 used a different body-block mask": that mask is this box.
- `02-arrays-dat.md` — the full layout of `arrays.dat` (25 arrays, nested kinds) and what each array is for.
- `03-cut-lift-chest.md` — sword cut, lift by glove power, chest open by tile and by record, the A-key order, probe offsets.
- `04-public-server-run.md` — what a public server sent on login (counts and layouts), why no chests were drawn, the weapon-script line-break byte, the capture link type.
- `open-questions.tsv` — what was not read.

Checked against: Seihai PRs 60 to 64 (tests listed there).

`client-internals/decomp-findings-2026-10-25-l/01-walking-body-box.md`

Walking: the wall test is a box, not a point

Date: 2026-10-25. Source: the 2.05 client's walking routine (derived).

The older note (`../foot-test.md`) measured a single-point test in an SDL reimplementation and said the official client "used a different body-block mask". This is that mask.

Step

- Each arrow key is a separate step, handled in the order up, left, down, right. A diagonal is two steps, each tested against the position the previous one left.
- A step is half a tile times a speed factor: 1.0 on foot, 1.2 with a high speed state. Other factors (swimming, carrying) exist; their values were not read here. The step vectors per facing are (0, -0.5), (-0.5, 0), (0, 0.5), (0.5, 0).

Six probe points per facing

Offsets from the sprite's top-left, evaluated at the position the step would reach:

facing	the points
up	y + 1.0; x + 0.525, 1.475, 1.525, 2.475, 2.525, 0.475
left	x + 0.5; y + 1.025, 1.975, 2.025, 2.975, 3.025, 0.975
down	y + 3.0; x + 0.525, 1.475, 1.525, 2.475, 2.525, 0.475
right	x + 2.5; y + 1.025, 1.975, 2.025, 2.975, 3.025, 0.975

That is a box x + 0.5 .. x + 2.5, y + 1 .. y + 3 (two tiles square), with the points on its leading edge. Points are truncated to a tile.

Rule

- The step is kept when the **first four** points are free.
- Otherwise the position is put back and the player is put in the "pushing" state (the push/pull block action starts from that state).
- On a refused step, a 0.1-tile nudge along the edge may apply so a corner does not catch. With points numbered 1..6: if point 1 is blocked and 3, 4, 5 are free, move +0.1 across the edge (right for up/down, down for left/right); if 1 and 2 are free, 4 is blocked and 6 is free, move -0.1.
- After the step the fractional part of the position is snapped to a tile edge when it lies within 0.0625 of it (not read in detail).

What a point tests

Blocked if any of: a chest record covers it (2 x 2, edges included, opened or not); a solid object sprite or another blocking sprite is there; the tile is neither in the passable list (`arrays.dat` array 0) nor in the liquid list (array 1). Lava is a liquid and walkable.

Other probes (same routine family)

- A-key probe, lift, and sword cut: one tile beyond the middle of the leading edge: up (x + 1.5, y), left (x - 0.5, y + 2), down (x + 1.5, y + 4), right (x + 3.5, y + 2).
- A separate probe set is used by the push/pull block action; not read here.

`client-internals/decomp-findings-2026-10-25-l/02-arrays-dat.md`

arrays.dat: layout and meaning

Date: 2026-10-25. From the client's loader and its uses (derived).

Layout

A run of arrays to the end of the file (the 2.22 file: 11,500 bytes, 25 arrays). Each array starts with a big-endian `u32` header:

- low 16 bits: the count `n`;
- high 16 bits: 0 = `n` big-endian `u32` ints follow; non-zero = `n` nested arrays follow, each with its own header.

The 2.22 file parses completely this way with no bytes left over. Counts of the 25 arrays: 934, 116, 133, 76, 40, 4, 4, 79, 4, 2, 2, 5, 5, 4, 2, 2, 2, 124, 8, 6, 32, 4, 5, 4, 356.

Arrays (index = position in the file)

index	what	shape
0	passable tile ids (`nowall`); any other tile is a wall	flat
1	liquid tile ids (water and lava)	flat
3	lava tile ids (liquids that are lava)	flat
9	cuttable 2 x 2 objects	nested: type -> column -> row

index	what	shape
10	the 2 x 2 tiles a cut leaves	same shape, same order
11	liftable 2 x 2 objects	nested
12	the 2 x 2 tiles a lift leaves	nested
13	closed chest tiles (1784 1785 1800 1801)	flat
14	open chest tiles (829 845 / 830 846)	nested

Others (2, 4 to 8, 15 to 24) are not read here; several hold per-tile lookups the client uses for drawing and sounds.

Objects are indexed `[column][row]`. The 2.22 tables: 2 cut types (the bush `2 3 / 18 19` and `420 421 / 436 437`, leaving `677 678 / 693 694` and `679 680 / 695 696`), and 5 lift types (the bush, a 512 block, a 684 block, the rock `34 35 / 50 51`, a 990 block) leaving `677 678 / 693 694`, `1802 1803 / 1818 1819`, `1770 1771 / 1786 1787`, and `1834 1835 / 1850 1851` twice.

Note: reading only the first four arrays as flat int lists (as a first loader did) is right for 0 to 3 and ignores everything else; arrays 9 to 12 need the nested reading.

`client-internals/decomp-findings-2026-10-25-l/03-cut-lift-chest.md`

Sword cut, lift, and chests

Date: 2026-10-25. From the 2.05 client (derived).

Matching a 2 x 2 object

Take the tile under the probe point. For each table entry in order, for each of its four tiles equal to that tile, the object's top-left is the point minus that tile's column and row. The four board tiles there must equal the entry's four, all on the board. First full match wins.

Sword cut

At the start of a sword swing, at the A-key probe point: match against the cut table (all entries, no glove). On a match the four tiles are replaced by the entry's "left" tiles (array 10) through the routine that also writes board changes the server relays, then a routine that takes the entry index spawns debris or items (not read).

Lift (A key)

First action of the A key (checked before pulling a block). Glove power `g` allows the first `g + 2` lift entries: a bare hand the first two (bush, 512 block), power 1 adds the 684 block, power 2 the rock and the 990 block. A match replaces the four tiles with the entry's "left" tiles (array 12) and the player carries an object. The throw, its flight and landing, and the carrying pose were not read.

Chests

The open actions below are what the Seihai client does after its reading of the 2.05 client; only the lift and cut probes were read in detail.

- By board tile: if the tile at the A-key probe is one of the closed chest tiles (1800/1801 bottom row), it is replaced by the open chest 2 x 2 and PLI_OPENCHEST is sent.

- By record: a `PLO_LEVELCHEST` record at the probe, not yet opened: it is set opened, the item is spawned, PLI_OPENCHEST is sent, and the sign the record names is shown.
- A chest record has **no draw code**. A chest looks like its board tiles and nothing else. The record adds the open action and a 2 x 2 solid footprint (opened or not).
- Record bytes (each plus 32): opened flag, x, y, and, only if not opened, item index and sign index. The item index names an entry of the client's built-in chest item list (the list was not read here).

A-key order

Lift is tried first, then grab (pull a block). Where reading a sign and opening a chest sit in that order was not read.

Sword while carrying

See `../decomp-findings-2026-10-28/01-sword-while-carrying.md`: with an object carried, the sword key throws it instead of swinging.

`client-internals/decomp-findings-2026-10-25-l/04-public-server-run.md`

A public GServer-RS login, 2.22 profile (2026-10-25)

Counts and layouts only. The capture, screenshots, and the server's address are not in this repository.

Login burst

819 messages in the session. In order, the ones that matter: own player props, flags, a weapon list of one weapon (`-gr_movement`, with its script), `SETACTIVELEVEL`, the level board (64 x 64 tile ids, two bytes each), three `PLO_LEVELCHEST` records, no `PLO_LEVELSIGN`, four `PLO_NPCPROPS` records, then text and window messages. One file request in the session was answered with a send-failed message.

Why no chests were drawn

All three chest records are the short form (opened, x, y): the server says the account had opened them. The board has none of the chest tile ids (closed 1784 1785 1800 1801 or open 829 830 845 846) and the three positions are plain grass. The client draws nothing for a chest record, so nothing is drawn.

Weapon scripts and line breaks

The server writes a line break in a weapon script as §. A latin-1 server sends the byte A7; a UTF-8 server sends C2 A7. Decoded as latin-1 without handling the pair, the script becomes one `//` comment line and does nothing. With the break read as a newline the `-gr_movement` script parses with no diagnostics. It sets two player strings from the position, rounded to half a tile, and sets the chat property to a fixed word, every 0.05 s.

Capture link type

A capture taken with `tcpdump -i any` uses Linux cooked v2 (pcap link type 276, a 20-byte pseudo-header whose first two bytes are the protocol type); older cooked captures use type 113 with 16 bytes.

`client-internals/decomp-findings-2026-10-25-m/README.md`

Decomp findings 2026-10-25 (m): the whole walking rule

Derived facts from reading the walking routine of the 2.05, 2.17.1, 2.22 and 2.31 clients. It is the same routine in all four: the same probe table, nudge table, branches and constants. No addresses, no function names, no decompiler output, no code. Numbers are tile units.

- `01-walking-rule.md` — step order, the six probe points and how a point is turned into a tile, the corner nudge, the snap flush to a wall after a refused step, the 0.0625 line snap, what a point can be blocked by. Completes `../decomp-findings-2026-10-25-l/01-walking-body-box.md`, which had the probe table and the nudge conditions but not the snaps or the tile rounding.
- `open-questions.tsv` — what was not read.

Why this matters: a port that implements only "refuse the step if the first four points are blocked" stops half a tile short of any wall on its right or below (the right and bottom probes sit on the tile line one tile beyond the body) and sticks on corners. Seihai had exactly that; see its `documentation/walking-collision.md`.

Checked against: Seihai's tests of the rule (stopping positions from both half-tile alignments, the slide round a wall's end, the line snap). No lab run of the real client yet.

`client-internals/decomp-findings-2026-10-25-m/01-walking-rule.md`

The walking rule, whole

Date: 2026-10-25. Source: the walking routine of the 2.05, 2.17.1, 2.22 and 2.31 clients (derived; the four agree on every table and constant below).

One key, one step

The routine is given the key's direction, 0 up, 1 left, 2 down, 3 right, and runs the step for that direction. With several keys held the steps run in the order up, left, down, right, each at the position the earlier ones left. The first thing it does is set the player's facing to that direction. A step is half a tile times a speed factor (1.0 on foot); the step vectors are (0, -0.5), (-0.5, 0), (0, 0.5), (0.5, 0). Positions are doubles, in tiles.

The probe points

Six points per facing, as offsets from the sprite's top-left, read at the position the step would reach (table as in `../decomp-findings-2026-10-25-l/ 01-walking-body-box.md`):

facing	the six points
up	y + 1.0; x + 0.525, 1.475, 1.525, 2.475, 2.525, 0.475
left	x + 0.5; y + 1.025, 1.975, 2.025, 2.975, 3.025, 0.975
down	y + 3.0; x + 0.525, 1.475, 1.525, 2.475, 2.525, 0.475
right	x + 2.5; y + 1.025, 1.975, 2.025, 2.975, 3.025, 0.975

Numbered 0 to 5 in that order. Each point goes through the same blocked test, which returns blocked or free.

Point to tile. A coordinate becomes a tile as `floor(v + 0.0001)`. A point exactly on a tile line reads the tile that starts there; a value a hair under a whole number (rounding) reads the tile above it. A tile outside 0..63 is not read from this level: it is the neighbour in a gmap, or a fixed tile outside a gmap.

What blocks a point. In this order: a list of objects with two flags and a body box like the player's (only when a client option is on); the level's NPCs that have a blocking shape or image; a list of 1.5 x 2 boxes with an enabled flag (x + 0.25 to x + 1.75, y to y + 2); the chests (each record x, y to x + 2, y + 2, edges included, opened or not); then the tile, which blocks unless its id is in one of the two passable lists of `arrays.dat` (arrays 0 and 1). The first list and the 1.5 x 2 list were not identified.

Decision

Points 0 to 3 are the step test. If all four are free the step is kept (and the position is set to the stepped one). If any is blocked it is refused.

On a refused step

In this order:

1. **Corner nudge.** If point 0 is blocked and points 2, 3 and 4 are free: add 0.1 across the edge (+x for up and down, +y for left and right). Else if points 0 and 1 are free, point 3 is blocked and point 5 is free: add -0.1. The nudge table is the same in all four clients: up and down `(+0.1, 0)` and `(-0.1, 0)`, left and right `(0, +0.1)` and `(0, -0.1)`. It is added to the stepped position.
2. **Take the step back** (subtract the step vector times the speed factor). The nudge stays.
3. **Flush snap, along the facing only**, if the fraction of the coordinate is above zero (the fraction is 0 for a value just under a whole number): up `y = floor(y)`; left `x = floor(x + 0.5) - 0.5`; down `y = floor(y + 1)`; right `x = floor(x + 1.5) - 0.5`. The body's edge ends on a tile line. There is no wall test on the snapped position.

When the player is standing on a platform object or has a parent, the snaps are relative to the parent's position; without one the offset is 0.

After every step

Accepted or refused: `y` snaps to `floor(y + 0.0625)` if the fraction of y is under 0.0625 or over 0.9375. The same for x on `x + 0.5`: if its fraction is out of that band, `x = floor(x + 0.5 + 0.0625) - 0.5`. Then the position is stored. (A refused step still stores the position: that is how the nudge and the flush snap move the player.)

What this means for stopping positions

- Moving left or up, the probe is inside the body box; a body ends with its edge on the wall's tile line without any snap.
- Moving right or down, the probe is the tile line one tile beyond the body. The step that would put the body's right or bottom edge on a wall's tile line is refused (its probe reads the wall tile), and the flush snap then moves the body that half tile. Without the snap the body would stop half a tile short.
- A 2-wide body fits a 2-tile gap and not a 1-tile gap.
- Pressing against the end of a wall slides the body round it at 0.1 tile per tick: ten ticks to clear one tile.

Not read

See `open-questions.tsv`.

`client-internals/decomp-findings-2026-10-28/01-sword-while-carrying.md`

Sword key while carrying

Date: 2026-10-28. From the 2.05 client (derived). 2.22 and 2.31 not read.

The game keys reach the player by index: weapon (default D), sword (default S) and action (default A).

- **Sword key with an object carried:** no swing. The carried object is thrown, the same throw the action key does: it leaves from above the head, the carry sprite goes back to none (-1) and the idle gani is set. If the player's mode was walking, or the lay-down mode below, the mode goes back to idle. No swing precondition (sword power, swimming) is checked on this path.
- **Sword key with empty hands:** the normal swing.
- **Action key with an object carried:** throws, except in the lay-down mode, which has its own action.
- **Weapon key with an object carried:** does not throw. It only matters when the carried sprite is 239, 261 or 263; then the player enters a lay-down mode with its own gani. What that mode does was not read.

Online, a throw also triggers the client's player-props send (the carry sprite change); the exact bytes were not captured.

Not confirmed in a lab capture yet.

`client-internals/decomp-findings-2026-10-28/02-item-list.md`

Item list (Q key)

Date: 2026-10-28. From the 2.05 client (derived). 2.22 and 2.31 not read.

- The game actions by index: 0 up, 1 left, 2 down, 3 right, 4 item, 5 attack, 6 action, 7 map, 8 switchtochat, 9 itemlist, 10 pause. `itemlist` (default Q) opens and closes the item list. It opens from the normal state only when a player flag allows it (flag not identified).
- The list holds the player's weapons: built-in weapons and NPC weapons. A `/` in a weapon name makes a folder; folders of the current level come first, as 32 x 32 cells. Names starting with `-` are skipped when selecting.
- Layout: a framed window from (16, 80) to 16 px from the right and bottom edges; cells inside from (48, 112) to 48 px from the right and 64 px from the bottom. Cell size: max(image width, label width) + 10 by image height
    - label height + 15; cells are packed in the first free place, left to right then down. Image centred with 5 px above; label centred below; no-image weapons get a fixed 32 x 32 placeholder. Selected cell filled RGB(255, 212, 128). Font size 16. Scroll arrows when cells are cut off.
- Arrow keys: stretch the selected cell to the edge in that direction and take the overlapping cell with the closest near edge. Moving selects the weapon immediately; nothing is sent. Item key or Enter on a folder opens it; attack key goes up a folder. Other game keys close the list; Esc closes it. A mouse click on a cell selects it.
- Delete on the selected weapon removes it; online it sends PLI_NPCWEAPONDEL (29) with the weapon name, or PLI_NPCDEL (22) with the NPC id for an NPC that is not a weapon.

`client-internals/decomp-findings-2026-10-28/03-edge-links.md`

Edge links and the big map

Date: 2026-10-28. From the 2.05 client (derived). 2.22 and 2.31 not read. Not checked against a running client.

The level's neighbours

When a level is entered, the client fills a 3 x 3 table of level names around it from the big map: the text grid of quoted level names that the map screen uses (`maps/bigmap.txt` style, one row per line). It finds the current level's name in the grid and copies the eight names around it; cells off the grid stay empty. This happens only while a big map is loaded, and is skipped under one global flag (not identified; it may be the gmap or editor state). For each non-empty neighbour name the client also looks up a loaded level object of that name; that slot stays empty when the level is not loaded.

So the big map is not only a picture for the client: it decides which levels are "next to" the current one.

Following a link

Each tick the client takes a probe point: the player's position plus a per-facing offset. Then for each link of the level, in order:

1. **West edge link**: the link's x is 0, its width is 1, and its destination is the west neighbour's name (exact text compare). It fires when the probe x is 0 or less while facing left, or the player x is below -2.5, or, when the west neighbour level is **not** loaded, when the probe is inside the link's rectangle. Arrival: **x = player x + 64**, y unchanged. The link's own new-x/new-y are not used.
2. **East edge link**: x 63, width 1, destination = east neighbour. Fires when the probe x is 64 or more while facing right, or the player x is above 63.5, or (east neighbour not loaded) the probe is in the rectangle. Arrival: **x = player x - 64**, y unchanged.
3. **North edge link**: y 0, height 1, destination = north neighbour. Probe y 0 or less facing up, or player y below -3.0, or (not loaded) in the rectangle. Arrival: **y = player y + 64**, x unchanged.
4. **South edge link**: y 63, height 1, destination = south neighbour. Probe y 64 or more facing down, or player y above 63.0, or (not loaded) in the rectangle. Arrival: **y = player y - 64**, x unchanged.
5. **Any other link** (including an edge link whose destination is not the neighbour on that side): it fires when the probe is in the rectangle, except that a probe within 1 tile of an edge (x or y at most 1, or at least 63) only counts while the player faces out across that edge. The arrival coordinates are the link's new-x and new-y, evaluated as expressions (`playerx` / `playery` give the player's current coordinate).

The first link that fires is taken; offline the client loads the level, online it asks the server.

So an edge link between two levels that sit side by side on the big map keeps the player's exact offset across the seam (east off x 63.6 arrives at x -0.4), whatever the link's new-x says. Without a big map, or for a level that is not the big-map neighbour, the link's own coordinates are used, and `playerx` on an east-edge link then arrives on the east edge of the destination.

Link records (for reference)

- A link line has the destination, x, y, width, height, and optionally new-x and new-y. A missing new-x or new-y, or one that is `-1`, means `playerx` / `playery`.

- The level editor's default for a new link on an edge: on the east edge (x 63) new-x is `0`; on the south edge (y 63) new-y is `0`; on the west edge (x 0, width 1) new-x is `61`; on the north edge (y 0, height 1) new-y is `61`. The other coordinate stays `playerx` / `playery`.

Open

- The probe offsets per facing were not read.
- The flag that turns the neighbour table off was not identified.
- Which big map a 2.05 client uses offline, and whether 2.22 does the same with the `PLO_BIGMAP` (171) map, is not read.
- How walking behaves past the edge when the neighbour level is loaded (the player x can go below 0 or above 63.5 before the link fires), and whether the neighbour's tiles are tested there, is not read.

`client-internals/decomp-findings-2026-10-28-b/README.md`

Decomp findings 2026-10-28 (b): tile types in 2.05

Derived facts from reading the 2.05 rev 2 client: how a tile's type is decided, and the script functions `tiletype`, `onwall` and `onwater`. No addresses, no function names, no decompiler output, no code.

- `01-tile-type-tests.md`: there is no type table for the default tileset. Each type test asks one `arrays.dat` list. Also covered: the fixed position rule for tilesets of the new kind, `tiletype()` returning -1 on the default tileset, and lava counting as water.
- `tile-type-lists.tsv`: which array or rule answers each type number.

Checked against: Seihai PR 108 (tests listed there). Static read only. Nothing was run against a server.

`client-internals/decomp-findings-2026-10-28-b/01-tile-type-tests.md`

Tile type tests (2.05 rev 2)

Date: 2026-10-28. From the client's per-type test, its wall test, the `arrays.dat` loader and the script expression handler (derived).

Which rule applies

Each level picks its tileset from the tile definitions (`addtiledef`). The type of the chosen definition decides how tile types are answered:

- **0, the default** (`pics1.png`): from the `arrays.dat` lists.
- **1 and 2, the new kind**: from the tile id alone, by a fixed rule.
- **5**: no board rule. Only NPC shapes have types.

An NPC shape that the player stands on (`setshape2` with tile types) answers first, for every tileset.

The default tileset

No 4096-entry type table is built. Each question asks one list, and looking up a tile is a binary search of a sorted list:

question	list
wall	not in array 0 (passable) and not in array 1 (liquid)
3 chair	array 4
4 bed top	array 5
5 bed bottom	array 6
11 water	array 1
12 lava	array 3
20 throw-through	array 2
21 jumping stone	array 8: four lists, one picked by the object's direction

Any other type number is answered only by NPC shapes, so a board tile is never type 0, 2 or 6 to 10 in this tileset.

Array 7 is not a type list. A tile's position in it (0–11, 12–19, 20–62, 63 and up) gives one of four ground classes, the same four that shape types 6 to 9 give. The horse uses the class to pick the sprite drawn under it.

Off the board, the tile reads as id 632. The wall test checks bounds first and calls off-board a wall. The type tests skip the bounds check and look up 632, which is in no list in the 2.22 file.

Consequences:

- **Lava passes the water test.** The liquid list holds lava too, so `onwater` and the swim test are both true on lava. A separate lava test counts ticks on lava. When the count passes 100, the client calls what looks like the hurt routine, and the count resets. Reading that call as the hurt routine is **inferred**.
- **The lava test in detail.** Once per player tick the client tests one point, the player's position plus (1.5, 2), against the lava list (array 3) only. On lava a counter goes up. On the 101st tick in a row it calls the hurt routine with -0.5 (half a heart) and a flag that skips the usual "can be hurt" checks, then sets the counter to 0. A tick off lava sets it to 0. Which of the two offsets belongs to x was read from the order of two property reads, so (1.5, 2) rather than (2, 1.5) is **inferred**. The tick length was not read here.
- **A tile can be in several lists.** In the 2.22 file, 6 of the 133 throw-through tiles are passable, and 9 of the 76 lava tiles are not in the liquid list. All 9 are in the passable list, so they are walked on, not swum in, and the lava test still burns on them. (An earlier version of this note called them walls.)
- **Deep water is in no list.** A level made for testing (2026-10-28) uses a dark water tile for deep water. In the 2.22 file that tile is in none of arrays 0 to 3, so the wall test calls it a wall: deep water blocks. Shallow water in the same level is in the passable list only, so it is walked on.
- A thrown object is stopped by a tile that is not type 20 and that the wall test calls a wall.

Tilesets of the new kind

With `row = id / 16` and `col = id % 16`, a tile is a **wall** if any of these holds:

- id 32
- row 26 or 27 and col 2 or more
- row 30 to 47, 84 to 95, or 116 to 127
- row 128 or more and `(row / 16)` odd

A wall tile is 20 if its id is 512 to 767, 21 if it is in row 30 or 31 with col 8 or more, and 22 otherwise.

A tile that is not a wall is checked in this order, and the first match wins:

1. 288–319 gives 2
2. 320–383 gives 3
3. 384–399 gives 4

4. 400–415 gives 5
5. 64–287 gives 11
6. 192–287 gives 12. This step can never be reached, because step 5 has already taken those ids.
7. row 26 or 27 with col 0 or 1, or 768–799, gives 6
8. 800–831 gives 7
9. 832–911 gives 8
10. 912–943 gives 9
11. 944–991 gives 10
12. anything else gives 0

Script functions

- `tiletype(x, y)` returns -1 unless the level's tileset is of type 1, 2 or 5. For type 5 it always returns 1. Otherwise it applies the rule above (an NPC shape's type wins when it is above 1). On the default tileset, scripts therefore always get -1.
- `onwall(x, y)` is the wall test.
- `onwater(x, y)` is the type 11 test, true on lava as well.

The 2.00 to 2.04 builds all carry the three names. Their code was not compared.